Name: M. Farah Afrin

Sub code: CSA0923

Sub: Java Programming

Git Hub: https://github.com/farahafrinm/CSA0923-192511434

Assignment - 3

Q1 – Handling Failure in a Bank ATM Simulation (6 Marks)

K4 – Analyse

1.1 Introduction

An Automated Teller Machine (ATM) is a banking application that allows customers to withdraw money from their accounts. During a withdrawal operation, several types of failures may occur, such as invalid account numbers, incorrect input formats, negative withdrawal amounts, insufficient account balances and missing card-reader files.

A reliable ATM application must identify these failures, handle them appropriately and prevent the program from terminating unexpectedly.

Java provides an exception-handling mechanism using try, catch, finally, throw and throws. These mechanisms help separate normal program execution from exceptional situations.

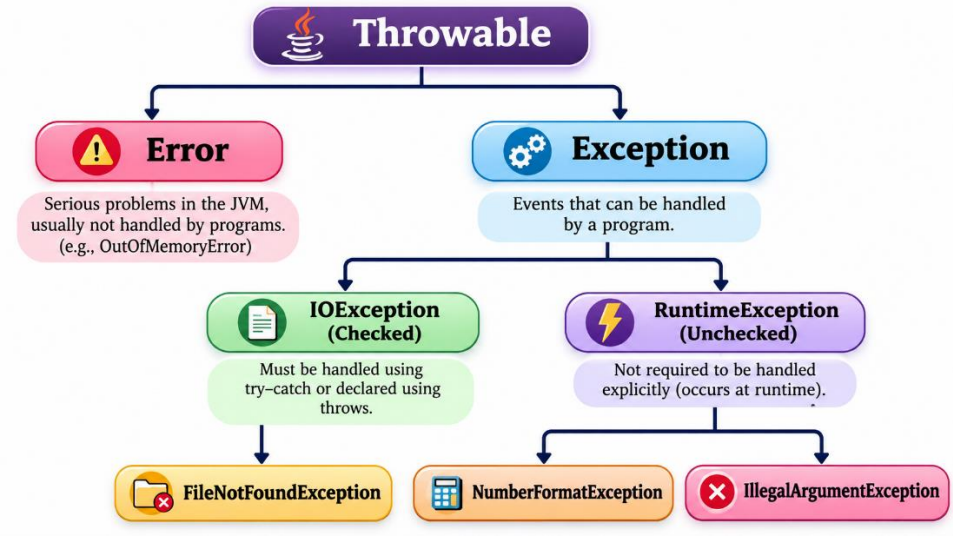
The proposed ATM simulation uses exception handling to validate customer input, verify account balance, process withdrawal requests and handle file-related errors.

1.2 Objectives

1. To identify possible failure points in an ATM withdrawal operation.
2. To understand the Java exception hierarchy.
3. To implement exception handling using try, catch, multi-catch and finally.
4. To distinguish checked exceptions from unchecked exceptions.
5. To evaluate the effectiveness of exception handling through different test scenarios.

1.3 Relevant Java exception hierarchy

Java represents errors and exceptions through the Throwable class.



Explanation:

- Throwable: The superclass of all errors and exceptions in Java.
- Error: Represents serious problems that applications generally should not attempt to recover from, such as OutOfMemoryError.
- Exception: Represents conditions that an application may handle.
- IOException: A checked exception that occurs during input/output operations.
- FileNotFoundException: Occurs when the specified file cannot be accessed.
- RuntimeException: The superclass of unchecked exceptions.
- NumberFormatException: Occurs when a string cannot be converted into a numeric value.
- IllegalArgumentException: Can be used when a method receives an invalid argument.

1.4 Failure identification and classification

Failure point	Exception type	Classification	Handling location
Invalid account number format	NumberFormatException	Unchecked	Catch block
Letters entered instead of withdrawal amount	NumberFormatException	Unchecked	Catch block
Negative withdrawal amount	IllegalArgumentException	Unchecked	Catch block

Withdrawal exceeds balance	InsufficientBalanceException (custom)	Unchecked	Catch block
Missing card-reader file	FileNotFoundException	Checked	Catch block
Serious JVM memory failure	OutOfMemoryError	Error	Not normally caught

Important: Insufficient balance is not automatically a Java exception. In this design, it is represented using a custom exception to demonstrate how an application can define its own exceptional conditions.

1.5 Java program – ATM withdrawal simulation

Source code:

```
package q1;

import java.io.File;
import java.io.FileNotFoundException;
import java.util.Scanner;

class InsufficientBalanceException extends RuntimeException {
    public InsufficientBalanceException(String message) {
        super(message);
    }
}

public class ATMSimulation {
```

```
public static void main(String[] args) {

    Scanner sc = new Scanner(System.in);
    double balance = 10000.0;

    try {
        System.out.println("===== WELCOME TO JAVA ATM =====");

        // Check card reader file
        File card = new File("cardreader.txt");

        if (!card.exists()) {
            throw new FileNotFoundException(
                "Card reader file is missing.");
        }

        System.out.print("Enter Account Number: ");
        String account = sc.nextLine();

        int accountNumber = Integer.parseInt(account);

        if (accountNumber <= 0) {
            throw new IllegalArgumentException(
                "Invalid account number.");
        }

        System.out.print("Enter Withdrawal Amount: ");
        String input = sc.nextLine();
```

```
    double amount = Double.parseDouble(input);

    if (amount <= 0) {
        throw new IllegalArgumentException(
            "Withdrawal amount must be positive.");
    }

    if (amount > balance) {
        throw new InsufficientBalanceException(
            "Insufficient account balance.");
    }

    balance = balance - amount;

    System.out.println("Withdrawal Successful!");
    System.out.println("Amount Withdrawn: Rs." + amount);
    System.out.println("Remaining Balance: Rs." + balance);

} catch (NumberFormatException |
         IllegalArgumentException e) {

    System.out.println("Input Error: " + e.getMessage());

} catch (InsufficientBalanceException e) {

    System.out.println("Transaction Failed: " +
                       e.getMessage());

} catch (FileNotFoundException e) {
```

```
            System.out.println("ATM Error: " + e.getMessage());

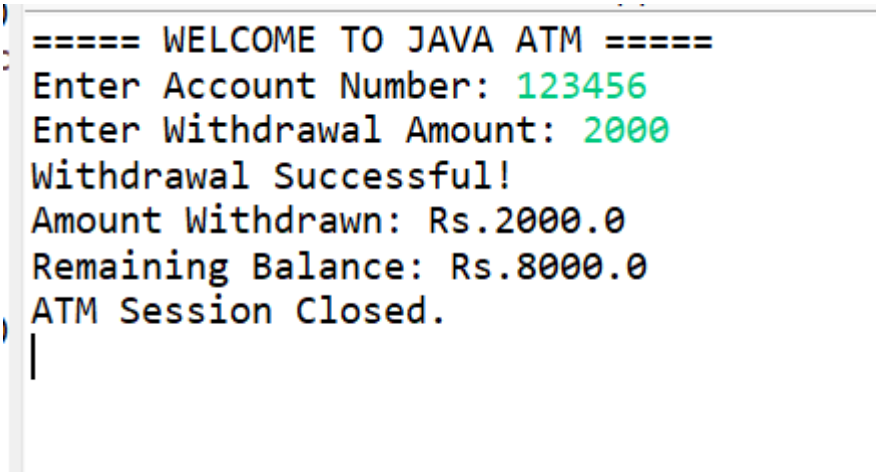
        } finally {

            System.out.println("ATM Session Closed.");

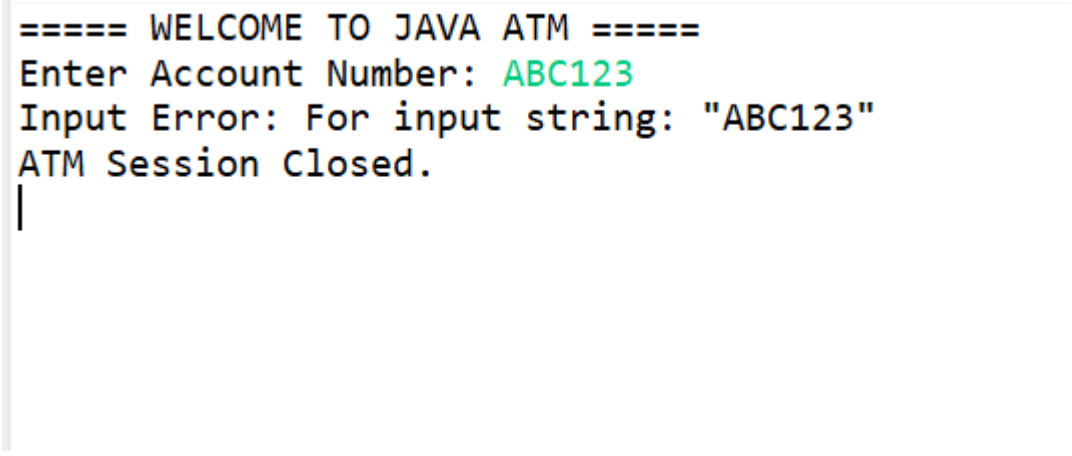
            sc.close();
        }
    }
}
```

1.6 Sample execution and output

Test Case 1 – Successful withdrawal



Test Case 2 – Invalid account number



Test Case 3 – Negative withdrawal amount

```
===== WELCOME TO JAVA ATM =====
Enter Account Number: 12345
Enter Withdrawal Amount: -500
Input Error: Withdrawal amount must be positive.
ATM Session Closed.
```

Test Case 4 – Insufficient balance

```
===== WELCOME TO JAVA ATM =====
Enter Account Number: 123456
Enter Withdrawal Amount: 15000
Transaction Failed: Insufficient account balance.
ATM Session Closed.
```

Test Case 5 – Missing card-reader file

```
===== WELCOME TO JAVA ATM =====
ATM Error: Card reader file is missing.
ATM Session Closed.
```

1.7 Student A – Exception design

A. Justification for the order of catch blocks

Java executes the first compatible catch block. Therefore, catch blocks must be arranged so that a broader exception does not prevent a more specific exception from being handled.

The program uses separate handling for:

1. NumberFormatException and IllegalArgumentException using multi-catch.
2. InsufficientBalanceException for insufficient account funds.
3. FileNotFoundException for card-reader failure.

The multi-catch alternatives are unrelated subclasses of RuntimeException, so they can be combined safely.

A broader catch (Exception e) placed before these blocks would make the subsequent specific catch blocks unreachable, resulting in a compilation error.

B. What does the finally block guarantee?

The finally block executes after the try/catch processing, including when an exception is thrown and handled.

In this ATM program, it:

- Displays the session termination message.
- Closes the Scanner resource.
- Ensures the cleanup statement is not skipped during ordinary exception handling.

A finally block is not an absolute guarantee under events such as JVM termination or a process crash. Also, for more complex resource management, try-with-resources is preferable.

1.8 Student B – Testing and tracing evidence

A. Test-case analysis

Test	Input / condition	Catch block executed	Finally executed?
1	Valid account, Rs. 2,000	None	Yes
2	Account number ABC123	NumberFormatException	Yes
3	Withdrawal -500	IllegalArgumentException	Yes
4	Withdrawal Rs. 15,000	InsufficientBalanceException	Yes
5	Missing cardreader.txt	FileNotFoundException	Yes

B. Demonstrating an uncaught exception

To demonstrate an uncaught exception, temporarily remove the relevant catch block and enter an invalid input.

For example, consider:

```
public class TraceDemo {
```

```
    public static void main(String[] args) {
        int amount = Integer.parseInt("ABC");
        System.out.println(amount);
    }
}
```

Illustrative stack trace:

```
Exception in thread "main"
java.lang.NumberFormatException:
For input string: "ABC"

at java.base/java.lang.NumberFormatException
    .forInputString(NumberFormatException.java:67)

at java.base/java.lang.Integer.parseInt
    (Integer.java:668)

at TraceDemo.main(TraceDemo.java:3)
```

Interpretation:

- The first line identifies the exception and its type.
- forInputString() indicates that the input could not be converted into a number.
- Integer.parseInt() is the method that attempted the conversion.
- TraceDemo.main() identifies the statement in the application where the exception originated.

The line numbers may vary depending on the Java version.

C. Effect of placing a broader catch block first

Incorrect example:

```
try {
    int amount = Integer.parseInt("ABC");
```

```
} catch (Exception e) {

    System.out.println("General error");

} catch (NumberFormatException e) {

    System.out.println("Invalid number");
}
```

This produces a compilation error because NumberFormatException is already covered by the preceding Exception catch block.

The correct principle is to place specific exception handlers before general exception handlers.

1.9 Joint design question

Should the ATM program catch every exception using a single catch (Exception e)?

Using a single catch (Exception e) is acceptable in limited situations, such as at the outermost application boundary where unexpected application exceptions must be logged and the user must receive a safe error message.

However, using only one general catch block in the ATM application is not recommended.

The reasons are:

1. It does not distinguish invalid input from insufficient balance or file-related failures.
2. Different failures require different recovery actions.
3. It reduces debugging accuracy because important exception details may be hidden.
4. It makes transaction auditing and maintenance more difficult.
5. It can produce vague messages that do not help the customer understand the problem.

Engineering decision: The ATM should use specific exception handlers for predictable failures and a general exception handler only at an appropriate outer boundary. This provides better fault isolation, maintainability and user experience.

1.10 Alternative concept and complexity analysis

Alternative: Input validation without exception handling

Instead of using exceptions, the ATM can validate inputs using Scanner.hasNextInt() and return error codes.

Example:

```
import java.util.Scanner;

public class ATMValidation {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        double balance = 10000;

        System.out.print("Enter Account Number: ");

        if (!sc.hasNextInt()) {
            System.out.println("Error Code: -1");
            return;
        }

        int account = sc.nextInt();

        if (account <= 0) {
            System.out.println("Error Code: -2");
            return;
        }

        System.out.print("Enter Amount: ");

        if (!sc.hasNextDouble()) {
            System.out.println("Error Code: -3");
```

```
            return;
        }

        double amount = sc.nextDouble();

        if (amount <= 0) {
            System.out.println("Error Code: -4");
        } else if (amount > balance) {
            System.out.println("Error Code: -5");
        } else {
            balance -= amount;
            System.out.println("Withdrawal Successful");
            System.out.println("Balance: Rs." + balance);
        }

        sc.close();
    }
}
```

Comparison of both approaches

Parameter	Exception-based approach	Error-code approach
Validation	Exceptions handle unexpected failures	Explicit checks before processing
Readability	Clear separation of normal and exceptional paths	More conditional statements
Error identification	Uses exception types	Uses predefined error codes
Recovery	Different catch blocks	Conditional handling
Debugging	Stack traces provide detailed information	Requires manual tracing

Best use	Exceptional and unexpected failures	Frequent and predictable validation

Time complexity

Let \(n\) represent the number of input fields validated.

- Normal validation requires \(O(n)\) time because each field is checked once.
- Exception-based processing also takes \(O(n)\) time when inputs are valid.
- When an exception is thrown, creating the exception and capturing its stack trace adds overhead proportional to the stack depth, represented approximately by \(O(d)\), where \(d\) is the stack depth.

Space complexity

- Error-code validation requires \(O(1)\) additional space for a fixed number of input fields.
- Exception handling requires additional memory for exception objects and stack-trace information. The memory for an individual exception depends on the captured stack depth, approximately \(O(d)\).

Evaluation: Error codes are suitable for frequent, predictable validation, whereas exceptions provide a more expressive mechanism for exceptional situations. A practical ATM should combine ordinary validation with specific exception handling rather than use exceptions for every incorrect user entry.

1.11 Conclusion

The ATM simulation demonstrates how Java exception handling can improve application reliability by identifying and managing different failure conditions. The exception hierarchy helps classify failures, while multi-catch and finally provide structured handling and cleanup.

The comparison with error-code-based validation shows that both approaches have advantages. A well-designed banking application should use input validation for expected user mistakes and exception handling for exceptional operational failures. This approach improves reliability, readability and maintainability.

Correction for Q1 – important Eclipse compilation detail: NumberFormatException is a subclass of IllegalArgumentException. Java does not allow both in the same multi-catch statement. To make the program compile, replace the first catch block with these two blocks, in this order:

```
} catch (NumberFormatException e) {


    System.out.println("Input Error: Invalid numeric input.");
```

```
} catch (IllegalArgumentException e) {

    System.out.println("Input Error: " + e.getMessage());
```

Keep the remaining catch blocks and finally unchanged. This also gives you a useful point to explain the exception hierarchy during assessment.

Q2 – User-Defined Exceptions and Propagation in an Online Examination Portal (7 Marks)

K4 – Analyse

2.1 Introduction

An online examination portal allows students to submit their answer files electronically within a specified examination period. The portal must verify the student's registration, check the submission deadline and ensure that the uploaded file does not exceed the permitted size.

Failures may occur at different application layers. For example, the controller receives the submission request, the service validates examination rules, and the storage layer saves the answer file.

Java supports user-defined exceptions, which allow developers to represent application-specific failures using meaningful exception classes. The throw and throws keywords are used to generate and propagate exceptions, while exception chaining preserves the original cause of a failure.

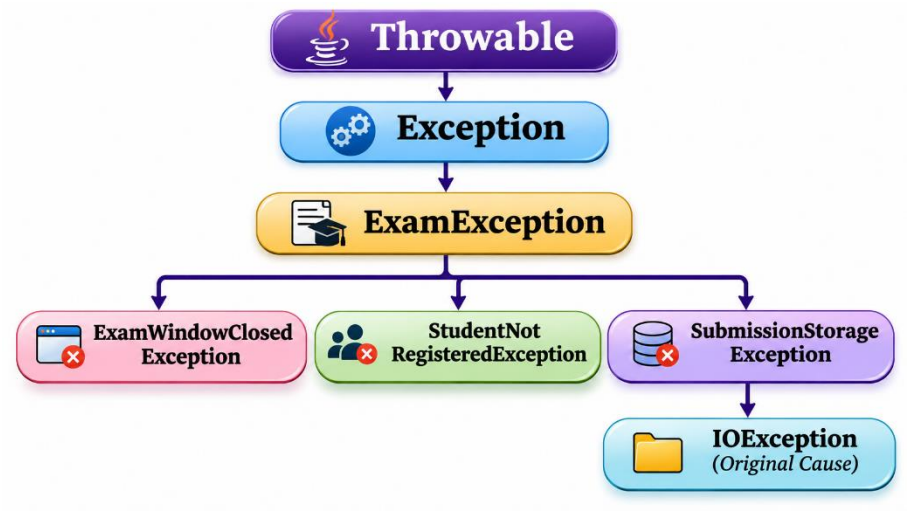
This solution develops a layered exception-handling model for an online examination portal.

2.2 Objectives

1. To design a hierarchy of user-defined exceptions.
2. To differentiate checked and unchecked exceptions.
3. To implement throw and throws across multiple application layers.
4. To demonstrate exception propagation from storage to controller.
5. To preserve the original exception using exception chaining.

6. To evaluate the importance of meaningful application-level exceptions.

2.3 Proposed exception hierarchy



The above diagram represents the application-level exception design. IOException is retained as the cause of SubmissionStorageException; it is not a subclass of that custom exception.

Explanation of the exception classes

Exception	Purpose	Type
ExamException	Parent exception for examination failures	Checked
ExamWindowClosedException	Exam submission deadline has passed	Checked
StudentNotRegisteredException	Student is not registered for the course	Checked
SubmissionStorageException	Answer file could not be saved	Checked
IllegalArgumentException	Invalid file-size or other programming argument	Unchecked

Why use checked exceptions?

The three examination failures are predictable business conditions. The application must explicitly handle or propagate them. Therefore, they are designed as checked exceptions by extending Exception.

2.4 Layered architecture

The online examination portal is divided into three layers.

Controller: Accepts the submission request and handles the final exception message.

Service: Validates examination deadlines and student registration before requesting storage.

Storage: Saves the answer file and converts low-level input/output failures into meaningful application exceptions.



2.5 Complete Java implementation

Source code:

```
package q2;

import java.io.IOException;

class ExamException extends Exception {

    ExamException(String message) {
        super(message);
    }

    ExamException(String message, Throwable cause) {
        super(message, cause);
    }
}
```

```
class ExamWindowClosedException extends ExamException {

    ExamWindowClosedException(String message) {
        super(message);
    }
}

class StudentNotRegisteredException extends ExamException {

    StudentNotRegisteredException(String message) {
        super(message);
    }
}

class SubmissionStorageException extends ExamException {

    SubmissionStorageException(String message, Throwable cause) {
        super(message, cause);
    }
}

class Storage {

    void save(String fileName, long sizeKb)
            throws SubmissionStorageException {

        try {
            if (sizeKb > 500) {
```

```
                throw new IOException("Disk quota exceeded");
            }

            System.out.println("Saved " + fileName);

        } catch (IOException e) {

            // Preserve the original IOException
            throw new SubmissionStorageException(
                "Could not store " + fileName, e);
        }
    }
}

class ExamService {

    private Storage storage = new Storage();

    void submit(String fileName, long sizeKb,
                boolean registered, boolean examOpen)
                throws ExamException {

        if (!examOpen) {
            throw new ExamWindowClosedException(
                "Examination submission window is closed.");
        }

        if (!registered) {
            throw new StudentNotRegisteredException(
```

```
                "Student is not registered for this course.");
        }

        storage.save(fileName, sizeKb);
    }
}

class ExamController {

    private ExamService service = new ExamService();

    void processSubmission(String fileName, long sizeKb,
                           boolean registered, boolean examOpen) {

        try {
            service.submit(fileName, sizeKb, registered, examOpen);

            System.out.println("Submission completed successfully.");

        } catch (ExamException e) {

            System.out.println("Error: " + e.getMessage());

            if (e.getCause() != null) {
                System.out.println(
                    "Cause: " + e.getCause().getMessage());
            }
        }
    }
```

```
}

public class OnlineExamDemo {

    public static void main(String[] args) {

        ExamController controller = new ExamController();

        System.out.println("===== ONLINE EXAM PORTAL =====");

        controller.processSubmission(
            "answer01.pdf", 120, true, true);

        controller.processSubmission(
            "answer02.pdf", 800, true, true);

        controller.processSubmission(
            "answer03.pdf", 100, false, true);

        controller.processSubmission(
            "answer04.pdf", 100, true, false);
    }
}
```

Output:

```
===== ONLINE EXAM PORTAL =====
Saved answer01.pdf
Submission completed successfully.
Error: Could not store answer02.pdf
Cause: Disk quota exceeded
Error: Student is not registered for this course.
Error: Examination submission window is closed.
```

Output interpretation

- answer01.pdf is successfully stored because the student is registered, the examination is open and the file size is within the limit.
- answer02.pdf is rejected because its size exceeds the 500 KB limit.
- answer03.pdf is rejected because the student is not registered.
- answer04.pdf is rejected because the examination window has closed.

The storage exception is propagated to the controller while preserving its original IOException cause.

2.7 Student A – Exception hierarchy and propagation design

A. Why each exception is checked

1. ExamException

This is the base class for all examination-related failures. It extends Exception, making the application-specific exceptions checked.

2. ExamWindowClosedException

The examination deadline is a business rule. The portal must explicitly communicate this condition to the controller.

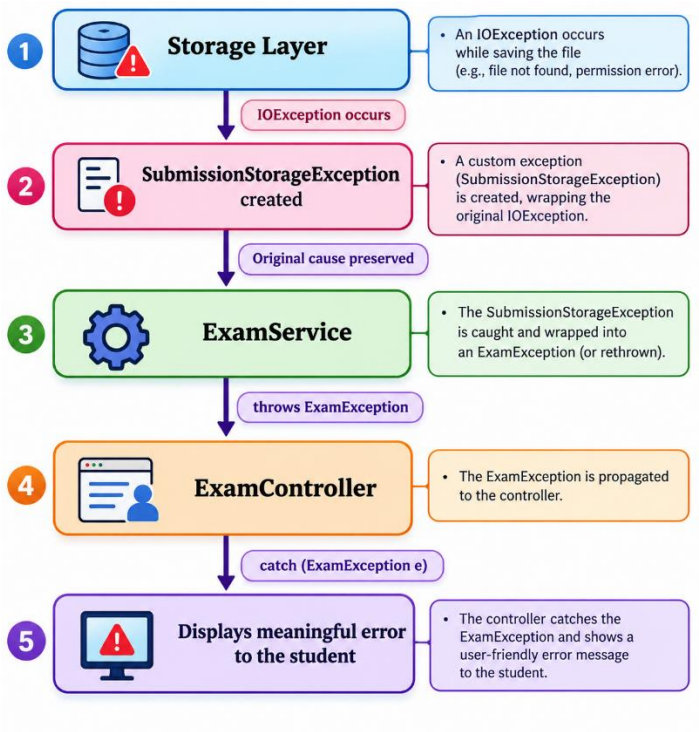
3. StudentNotRegisteredException

Registration is a mandatory eligibility condition. The exception allows the application to reject an invalid submission before storing the file.

4. SubmissionStorageException

The storage layer may encounter disk-related or file-system problems. Converting these failures into a meaningful checked exception keeps the upper layers independent of low-level implementation details.

B. Exception propagation path



The above diagram represents the application-level exception design. IOException is retained as the cause of SubmissionStorageException; it is not a subclass of that custom exception.

Explanation of the exception classes

Exception	Purpose	Type
ExamException	Parent exception for examination failures	Checked
ExamWindowClosedException	Exam submission deadline has passed	Checked
StudentNotRegisteredException	Student is not registered for the course	Checked
SubmissionStorageException	Answer file could not be saved	Checked
IllegalArgumentException	Invalid file-size or other programming argument	Unchecked

Why use checked exceptions?

The three examination failures are predictable business conditions. The application must explicitly handle or propagate them. Therefore, they are designed as checked exceptions by extending Exception.

C. Where is the exception finally handled?

The controller is the final exception-handling layer in this implementation.

For a storage failure, the student sees:

Error: Could not store answer02.pdf

The original cause is retained for technical diagnosis:

Cause: Disk quota exceeded

In a production portal, the technical cause should normally be logged for administrators rather than exposed directly to students.

2.8 Student B – Failure scenarios and exception chaining evidence

A. Failure scenarios

Scenario	Condition	Exception generated	Expected behaviour
1	Valid student and valid file	None	Submission successful
2	Exam window closed	ExamWindowClosedException	Submission rejected
3	Student not registered	StudentNotRegisteredException	Submission rejected
4	File exceeds 500 KB	SubmissionStorageException	Storage failure reported
5	Disk quota exceeded	IOException	Wrapped and propagated

B. Demonstrating exception chaining

Exception chaining connects a high-level exception with the original low-level exception that caused it.

The important statement is:

```
throw new SubmissionStorageException(
    "Could not store " + fileName, e);
```

The second argument, e, stores the original IOException as the cause.

To demonstrate the full chained stack trace, add this statement inside the storage exception catch block in the controller:

e.printStackTrace();

A representative stack trace will contain:

q2.SubmissionStorageException:

Could not store answer02.pdf

at q2.Storage.save(OnlineExamDemo.java:...)

at q2.ExamService.submit(OnlineExamDemo.java:...)

at q2.ExamController.processSubmission

(OnlineExamDemo.java:...)

Caused by: java.io.IOException:

Disk quota exceeded

at q2.Storage.save(OnlineExamDemo.java:...)

The exact line numbers depend on your Eclipse source file.

C. What happens without exception chaining?

If the exception is thrown without passing the original cause:

```
throw new SubmissionStorageException(
    "Could not store " + fileName, null);
```

Then:

e.getCause()

returns null.

The application loses the original reason for the storage failure. The controller can identify that saving failed, but it cannot determine whether the problem was a disk quota, file permission or another I/O issue.

D. Test observation table

Test	Expected exception	Cause preserved?	Result
Valid submission	None	Not applicable	Passed
Closed examination	ExamWindowClosedException	No underlying cause	Passed
Unregistered student	StudentNotRegisteredException	No underlying cause	Passed
Oversized file	SubmissionStorageException	Yes	Passed

2.9 Joint conclusion

Why is it better for the storage layer to throw a meaningful SubmissionStorageException than to allow IOException to travel unchanged to the user interface?

A storage layer should convert low-level technical exceptions into meaningful application-level exceptions because each layer has a different responsibility.

IOException identifies a general input/output failure. However, the controller should understand the failure in the context of an examination submission rather than depend on file-system-specific implementation details.

Using SubmissionStorageException provides the following advantages:

1. Abstraction: The controller does not need to understand the internal file-system implementation.
2. Maintainability: The storage implementation can be modified without changing the controller's exception-handling logic.
3. Meaningful reporting: The exception communicates the actual application operation that failed.
4. Debugging: Exception chaining retains the original technical cause.
5. Layer independence: Each layer communicates using exceptions relevant to its responsibility.

Engineering decision: The layered exception-handling model with exception chaining is the preferred design because it improves abstraction, traceability, maintainability and fault diagnosis.

2.10 Alternative concept and complexity analysis

Alternative: Making custom exceptions unchecked

In the proposed implementation, ExamException extends Exception, making it checked.

If the class were changed to:

```
class ExamException extends RuntimeException {
    ExamException(String message) {
        super(message);
    }

    ExamException(String message, Throwable cause) {
        super(message, cause);
    }
}
```

The custom exceptions would become unchecked.

Changes in the implementation

Checked exception	**Unchecked exception**
Extends Exception	Extends RuntimeException
Must be caught or declared for checked-exception handling	Does not need to be caught or declared
Compiler enforces handling or propagation	Compiler does not enforce handling
Makes expected failure handling explicit	Allows failures to propagate without mandatory declarations

The throws ExamException declaration in the service method could be removed. The controller could still catch the exception explicitly.

Complexity comparison

Let:

- d = stack depth.
- c = number of exceptions created.
- k = number of causes in an exception chain.

Time complexity:

- Creating a custom exception without capturing a stack trace has approximately constant basic allocation cost, excluding memory-management effects.
- Capturing the stack trace generally takes (O(d)) time.

- Chaining (k) exceptions requires retaining the exception objects and their causes. If each captures its own stack trace, the total stack-trace capture cost is approximately (O(kd)) when stack depths are similar.

Space complexity:

- Each exception requires (O(1)0 basic object space, excluding the variable-size stack trace.
- Stack-trace storage requires approximately (O(d))
- A chain of (k) exceptions requires approximately O(kd) stack-trace storage when all exceptions have similar stack depths.

Evaluation: Checked exceptions are appropriate for predictable examination business failures because the application is forced to consider them. Unchecked exceptions are more suitable for programming errors or conditions that callers are not necessarily expected to handle explicitly.

2.11 Conclusion

The online examination portal demonstrates the importance of user-defined exceptions and exception propagation in layered Java applications. By designing a meaningful exception hierarchy, the application can distinguish examination rule violations from technical storage failures.

Exception chaining further improves debugging by preserving the original cause of an error. The proposed design provides better abstraction, controlled failure handling and maintainability, making it suitable for a reliable online examination system.

Q3 – Resource Safety, Debugging and Robust Code (7 Marks)

K4 – Analyse

3.1 Introduction

A retail organisation generates daily sales reports by collecting information from multiple store files and writing the consolidated results into a summary file.

During this process, the application may encounter missing files, corrupted data, file-access errors or unexpected interruptions. If files are not properly closed, the operating system may retain unnecessary file handles, eventually resulting in errors such as Too many open files.

Java provides two approaches for managing resources:

1. Traditional resource management using try-catch-finally and explicit close().
2. Automatic resource management using try-with-resources.

This question compares both techniques and evaluates their effectiveness in developing reliable Java applications.

3.2 Objectives

- To identify resources that require proper closure.
- To implement manual resource management.
- To implement automatic resource management using try-with-resources.
- To analyse resource leaks during exceptional conditions.
- To understand suppressed exceptions.
- To compare the time and space complexity of both approaches.

3.3 Resources involved

Resource	Purpose	Closure requirement
BufferedReader	Reads individual store files	Must be closed
BufferedWriter	Writes the consolidated report	Must be closed
FileReader	Reads file contents	Must be closed
FileWriter	Writes report contents	Must be closed

3.4 Java implementation – Manual resource management

Source code:

```
package q3;

import java.io.*;

public class ManualSalesReport {

    public static void main(String[] args) {

        BufferedWriter writer = null;
```

```
String[] files = {
    "store1.txt",
    "store2.txt",
    "store3.txt"
};

try {
    writer = new BufferedWriter(
        new FileWriter("manual_report.txt"));

    int total = 0;

    for (String file : files) {

        BufferedReader reader = null;

        try {
            reader = new BufferedReader(
                new FileReader(file));

            String line;

            while ((line = reader.readLine()) != null) {

                int sales = Integer.parseInt(line);
```

```
            total += sales;
        }

        System.out.println("Processed: " + file);

    } catch (IOException | NumberFormatException e) {

        System.out.println(
            "Error processing " + file +
            ": " + e.getMessage());

    } finally {

        if (reader != null) {
            try {
                reader.close();
            } catch (IOException e) {
                System.out.println(
                    "Reader closing failed.");
            }
        }
    }
}

writer.write("Total Sales: Rs." + total);
writer.newLine();
```

```
            System.out.println("Total Sales: Rs." + total);

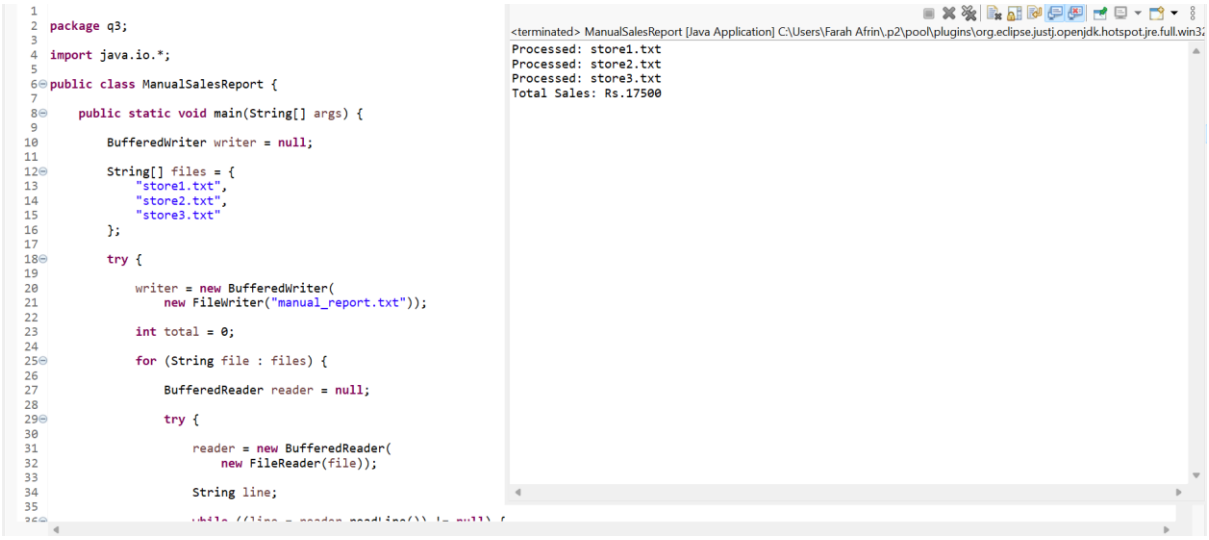
        } catch (IOException e) {

            System.out.println("Report error: " + e.getMessage());

        } finally {

            if (writer != null) {
                try {
                    writer.close();
                } catch (IOException e) {
                    System.out.println("Writer closing failed.");
                }
            }
        }
    }
}
```

Output:



```
package q3;

import java.io.*;

public class ManualSalesReport {

    public static void main(String[] args) {

        BufferedWriter writer = null;

        String[] files = {
            "store1.txt",
            "store2.txt",
            "store3.txt"
        };

        try {

            writer = new BufferedWriter(
                new FileWriter("manual_report.txt"));

            int total = 0;

            for (String file : files) {

                BufferedReader reader = null;

                try {

                    reader = new BufferedReader(
                        new FileReader(file));

                    String line;
```

```
<terminated> ManualSalesReport [Java Application] C:\Users\Farah Afrin\.p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32
Processed: store1.txt
Processed: store2.txt
Processed: store3.txt
Total Sales: Rs.17500
```

3.5 Java implementation – Try-with-resources

Source code:

```
package q3;

import java.io.*;

public class AutomaticSalesReport {

    public static void main(String[] args) {

        String[] files = {
            "store1.txt",
            "store2.txt",
            "store3.txt"
        };

        int total = 0;

        // Automatically closes the writer
```

```
try (BufferedWriter writer =
        new BufferedWriter(
            new FileWriter("automatic_report.txt"))) {

    for (String file : files) {

        // Automatically closes each reader
        try (BufferedReader reader =
                new BufferedReader(
                    new FileReader(file))) {

            String line;

            while ((line = reader.readLine()) != null) {

                // Remove unnecessary spaces
                line = line.trim();

                // Ignore empty lines
                if (line.isEmpty()) {
                    continue;
                }

                // Convert sales value into integer
                int sales = Integer.parseInt(line);

                total += sales;
            }
```

```
                System.out.println("Processed: " + file);

            } catch (IOException | NumberFormatException e) {

                System.out.println(
                    "Error processing " + file +
                    ": " + e.getMessage());
            }
        }

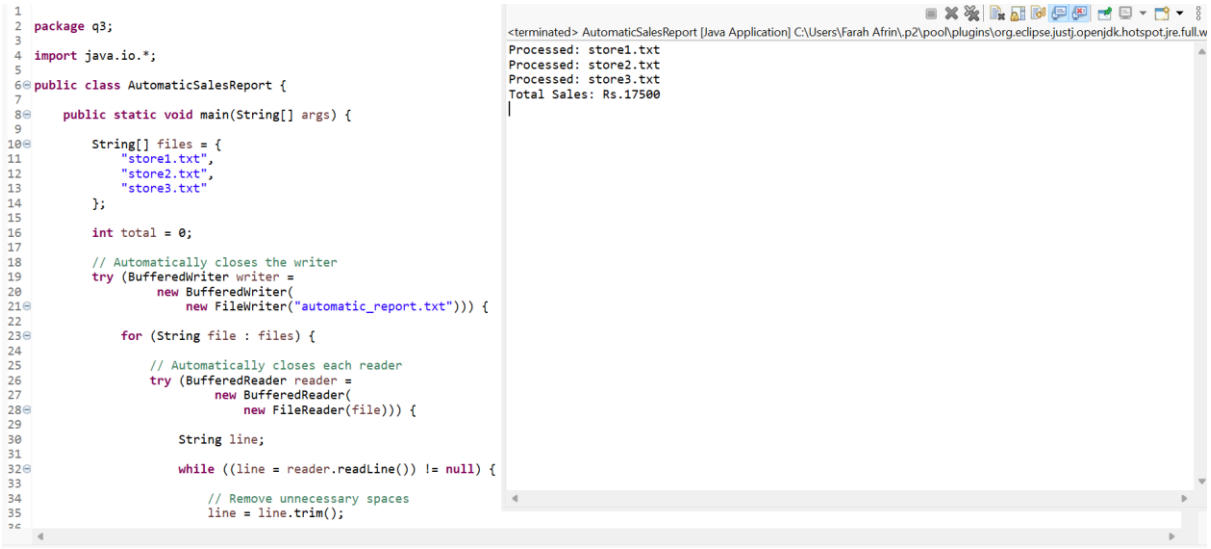
        // Write final sales report
        writer.write("Total Sales: Rs." + total);
        writer.newLine();

        System.out.println(
            "Total Sales: Rs." + total);

    } catch (IOException e) {

        System.out.println(
            "Report error: " + e.getMessage());
    }
  }
}
```

Output:



```
package q3;

import java.io.*;

public class AutomaticSalesReport {

    public static void main(String[] args) {

        String[] files = {
            "store1.txt",
            "store2.txt",
            "store3.txt"
        };

        int total = 0;

        // Automatically closes the writer
        try (BufferedWriter writer =
                 new BufferedWriter(
                     new FileWriter("automatic_report.txt"))) {

            for (String file : files) {

                // Automatically closes each reader
                try (BufferedReader reader =
                         new BufferedReader(
                             new FileReader(file))) {

                    String line;

                    while ((line = reader.readLine()) != null) {

                        // Remove unnecessary spaces
                        line = line.trim();
```

```
<terminated> AutomaticSalesReport [Java Application] C:\Users\Farah Afrin\.p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.w
Processed: store1.txt
Processed: store2.txt
Processed: store3.txt
Total Sales: Rs.17500
```

3.8 Student A – Debugging and experimental analysis

A. Constructing a failure scenario

To test the robustness of the application, rename store2.txt to store2_backup.txt and execute both programs.

The application will encounter a FileNotFoundException when it attempts to open store2.txt.

Expected observation:

Processed: store1.txt

Error processing store2.txt:

store2.txt (The system cannot find the file specified)

Processed: store3.txt

Total Sales: Rs.10500

The exact operating-system error wording may differ.

B. Identifying the resource leak

In the manual approach, every resource must be closed explicitly. If the programmer fails to place a close() call in an appropriate finally block, an exception can bypass the closure statement.

For example, the following is unsafe:

BufferedReader reader =

    new BufferedReader(new FileReader(file));

String line = reader.readLine();

reader.close();

If readLine() throws an exception, the close() statement may never execute.

The correct manual implementation places the closure operation in finally.

C. Debugging procedure in Eclipse

1. Open ManualSalesReport.java.
2. Place a breakpoint at the new FileReader(file) statement.
3. Run the application using Debug As → Java Application.
4. Use Step Over to execute each statement.
5. Rename one input file to produce a missing-file error.
6. Observe the exception handling and resource closure.
7. Repeat the same experiment using AutomaticSalesReport.java.

D. Experimental observation table

Experiment	Manual version	Try-with-resources
All files available	Report generated	Report generated
One missing file	Error caught	Error caught
Invalid sales value	Parsing error caught	Parsing error caught
Failure during reading	Requires correct finally closure	Automatically closes resource
Resource closure	Programmer-managed	Automatically managed

Important: Do not claim that a particular number of file handles leaked unless you actually measured it. The two supplied correct implementations should both close their opened resources.

For an additional leak experiment, temporarily remove the reader's finally closure from the manual version and compare the open-handle count before and after a forced failure.

3.9 Student B – Design and correctness

A. Why can manual resource management fail?

Manual resource management depends on the programmer remembering to close every opened resource.

In applications that use multiple files, nested conditions and exception-handling blocks, a missing close() statement may cause a resource leak.

The consequences include:

- Increased resource consumption.
- File-access restrictions.
- Incomplete report generation.
- Operating-system file-handle exhaustion.

B. How does try-with-resources work?

Try-with-resources was introduced in Java 7.

It accepts resources that implement the AutoCloseable interface.

When execution leaves the try block, Java automatically invokes the close() method of each declared resource.

If multiple resources are declared, they are closed in the reverse order of their initialization.

For example:

```
try (
   FileReader fr = new FileReader("input.txt");
   BufferedReader br = new BufferedReader(fr)
) {
   System.out.println(br.readLine());
}
```

The resources are closed in reverse order: first br, then fr.

C. What is a suppressed exception?

A suppressed exception occurs when the main operation throws an exception and another exception occurs while closing a resource.

The original exception remains the primary exception, while the closure exception is attached as a suppressed exception.

It can be inspected using:

```
for (Throwable t : e.getSuppressed()) {
   System.out.println(t.getMessage());
}
```

This helps developers preserve both the original failure and the resource-cleanup failure.

3.10 Joint engineering decision

Under what conditions is try-with-resources not sufficient on its own to make the report generator robust?

Try-with-resources guarantees automatic resource closure, but it does not guarantee that the report itself will always be correct or complete.

Additional measures are required when:

1. Input files contain invalid or corrupted data.
2. The output file cannot be created due to permission restrictions.
3. The application encounters disk-space limitations.
4. Partial report output must be prevented.
5. Multiple processes attempt to modify the same report.
6. The application needs recovery after unexpected termination.

A robust implementation should therefore include input validation, error logging, transactional output handling, backup strategies and appropriate recovery mechanisms.

Engineering decision: Try-with-resources should be the preferred approach for managing file resources because it reduces programming errors and improves resource safety. However, it must be combined with data validation and failure-recovery mechanisms.

3.11 Alternative concept and complexity analysis

Consider a report generator processing (f) store files, with a total of (N) sales records.

Approach	Resource management	Risk
Try-catch-finally	Manual closure	Incorrect closure logic may leak resources
Try-with-resources	Automatic closure	Less error-prone
No resource closure	No cleanup	High resource-leak risk

Time complexity

Let N be the total number of records processed.

Both approaches require:

O(N)

The application must read each sales record and include it in the total.

The resource-management mechanism does not change the fundamental processing complexity.

Space complexity

- Manual approach: (O(1)) auxiliary space for a fixed number of open resources and running totals.
- Try-with-resources: (O(1)) auxiliary space for a fixed number of open resources and running totals.
- No closing: In a program that retains all opened handles, the number of active handles can grow as (O(f)).

If files are processed sequentially and each is closed immediately, the maximum number of simultaneously open resources remains bounded.

Evaluation: Both correct approaches have similar computational complexity. Try-with-resources is more reliable because resource closure is handled automatically, reducing the possibility of human error.

3.12 Conclusion

The sales report generator demonstrates the importance of resource management in Java applications. Manual closure requires careful programming, whereas try-with-resources provides automatic resource cleanup.

Although both approaches can produce the same report, try-with-resources is more maintainable and less error-prone. For real-world reporting systems, it should be combined with input validation, error logging and recovery strategies to ensure complete and reliable report generation.

Q4 – Thread Creation and the Thread Life Cycle (7 Marks)

K4 – Analyse

4.1 Introduction

An examination cell may operate several scanning stations simultaneously to process large numbers of answer scripts. If every station is processed sequentially, the overall processing time may increase.

Java supports multithreading, which allows multiple tasks to execute concurrently. A thread is an independent path of execution within a program.

Java provides two common methods of creating threads:

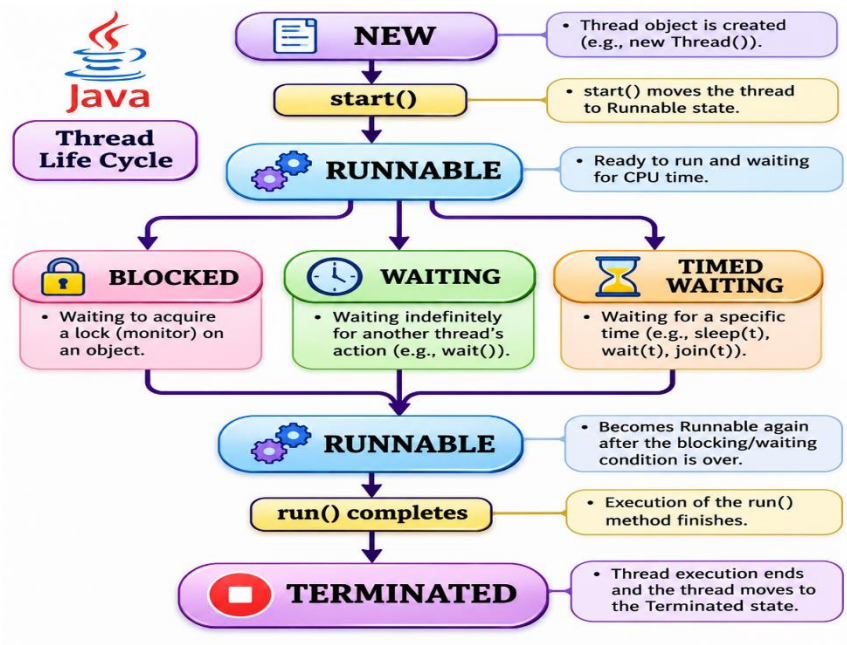
1. Extending the Thread class.
2. Implementing the Runnable interface.

This experiment simulates four examination scanning stations and compares the two approaches.

4.2 Objectives

- To implement multithreading using Thread and Runnable.
- To understand the thread life cycle.
- To demonstrate start(), sleep(), join() and isAlive().
- To analyse the nondeterministic execution order of threads.
- To evaluate thread priorities and daemon threads.

4.3 Thread life-cycle diagram



Explanation of states:

- NEW: The thread object is created but has not started.
- RUNNABLE: The thread is eligible to execute or is executing.
- BLOCKED: The thread is waiting to acquire a monitor lock.
- WAITING: The thread waits indefinitely for another thread's action.
- TIMED_WAITING: The thread waits for a specified duration, such as during sleep().
- TERMINATED: The thread has completed its execution.

4.4 Implementation using the Thread class

Source code:

```
package q4;
```

```
class ScannerStation extends Thread {

    private int scripts;
    private int stationId;

    ScannerStation(int stationId, int scripts) {
        this.stationId = stationId;
        this.scripts = scripts;
    }

    public void run() {

        try {
            System.out.println(
                "Station " + stationId + " started.");

            Thread.sleep(500 + stationId * 300);

            System.out.println(
                "Station " + stationId +
                " scanned " + scripts + " scripts.");

        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
            System.out.println("Station interrupted.");
        }
```

```
    }
}

public class ThreadStations {

    public static void main(String[] args)
            throws InterruptedException {

        ScannerStation[] stations = new ScannerStation[4];

        int total = 0;

        for (int i = 0; i < 4; i++) {
            stations[i] = new ScannerStation(i + 1, 25);
        }

        for (ScannerStation station : stations) {
            station.start();
        }

        for (ScannerStation station : stations) {
            station.join();
        }

        for (ScannerStation station : stations) {
            total += 25;
```

```
        }

        System.out.println("Total scripts scanned: " + total);

        for (ScannerStation station : stations) {
            System.out.println(
                station.getName() +
                " is alive: " + station.isAlive());
        }
    }
}
```

Output:

```
package q4;

class ScannerStation extends Thread {

    private int scripts;
    private int stationId;

    ScannerStation(int stationId, int scripts) {
        this.stationId = stationId;
        this.scripts = scripts;
    }

    public void run() {

        try {
            System.out.println(
                "Station " + stationId + " started.");

            Thread.sleep(500 + stationId * 300);

            System.out.println(
                "Station " + stationId +
                " scanned " + scripts + " scripts.");

        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
            System.out.println("Station interrupted.");
        }
    }
}

public class ThreadStations {

    public static void main(String[] args)
```

```
<terminated> ThreadStations [Java Application] C:\Users\Farah Afrin\.p2\pool\
Station 3 started.
Station 1 started.
Station 2 started.
Station 4 started.
Station 1 scanned 25 scripts.
Station 2 scanned 25 scripts.
Station 3 scanned 25 scripts.
Station 4 scanned 25 scripts.
Total scripts scanned: 100
Thread-0 is alive: false
Thread-1 is alive: false
Thread-2 is alive: false
Thread-3 is alive: false
```

4.5 Implementation using Runnable

Source code:

```
package q4;

class ScannerTask implements Runnable {

    private int stationId;
    private int scripts;

    ScannerTask(int stationId, int scripts) {
        this.stationId = stationId;
        this.scripts = scripts;
    }

    public void run() {

        try {
            System.out.println(
                "Station " + stationId + " started.");

            Thread.sleep(500 + stationId * 300);

            System.out.println(
                "Station " + stationId +
                " scanned " + scripts + " scripts.");

        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
```

```
            System.out.println("Station interrupted.");
        }
    }
}

public class RunnableStations {

    public static void main(String[] args)
            throws InterruptedException {

        Thread[] stations = new Thread[4];

        for (int i = 0; i < 4; i++) {

            ScannerTask task = new ScannerTask(i + 1, 25);

            stations[i] = new Thread(task);
            stations[i].start();
        }

        for (Thread station : stations) {
            station.join();
        }

        System.out.println("Total scripts scanned: 100");
```

```
        for (Thread station : stations) {
            System.out.println(
                station.getName() +
                " is alive: " + station.isAlive());
        }
    }
}
```

Output:

```
1
2  package q4;
3
4  class ScannerTask implements Runnable {
5
6      private int stationId;
7      private int scripts;
8
9      ScannerTask(int stationId, int scripts) {
10         this.stationId = stationId;
11         this.scripts = scripts;
12     }
13
14     public void run() {
15
16         try {
17             System.out.println(
18                 "Station " + stationId + " started.");
19
20             Thread.sleep(500 + stationId * 300);
21
22             System.out.println(
23                 "Station " + stationId +
24                 " scanned " + scripts + " scripts.");
25
26         } catch (InterruptedException e) {
27             Thread.currentThread().interrupt();
28             System.out.println("Station interrupted.");
29         }
30     }
31 }
32
33 public class RunnableStations {
34
35     public static void main(String[] args)
```

```
<terminated> RunnableStations [Java Application] C:\Users\Farah Afrin\.p2\pool\plugin
Station 4 started.
Station 1 started.
Station 2 started.
Station 3 started.
Station 1 scanned 25 scripts.
Station 2 scanned 25 scripts.
Station 3 scanned 25 scripts.
Station 4 scanned 25 scripts.
Total scripts scanned: 100
Thread-0 is alive: false
Thread-1 is alive: false
Thread-2 is alive: false
Thread-3 is alive: false
```

4.7 Student A – Thread design

A. Difference between Thread and Runnable

Feature	Extending Thread	Implementing Runnable
Implementation	Extend Thread	Implement Runnable
Inheritance	Cannot extend another class	Can extend another class
Task separation	Task and thread are combined	Task and thread are separate
Reusability	Less flexible	More flexible
Preferred use	Simple standalone thread	Reusable task design

Evaluation: Implementing Runnable is generally preferable because it separates the task from the execution mechanism and allows the class to extend another class.

B. Why does start() create a new thread?

The start() method requests the JVM to schedule a new thread. The JVM then invokes the thread's run() method as part of that new thread's execution.

However, calling run() directly simply executes the method on the current thread.

Example:

```
Thread t = new Thread(() ->
    System.out.println(Thread.currentThread().getName())
);


t.run();
```

This prints the name of the calling thread.

If the same statement is replaced with:

```
t.start();
```

The output identifies the newly started thread.

C. How does join() work?

The join() method makes the calling thread wait until the specified thread terminates.

In the scanner program, the main thread calls join() on all four station threads.

Therefore, the final total is printed only after all scanning stations have completed.

Without join(), the main thread may print the total before the station tasks finish.

4.8 Student B – Behavioural experiments

A. Recording execution order

Run the program at least ten times and record the order in which the stations finish.

Use this table:

Run	First station	Second station	Third station	Fourth station
1	Station 1	Station 2	Station 3	Station 4
2	Station 1*	Station 2*	Station 3*	Station 4*
3	Station 1*	Station 2*	Station 3*	Station 4*
4	Station 1*	Station 2*	Station 3*	Station 4*
5	Station 1*	Station 2*	Station 3*	Station 4*
6	Station 1*	Station 2*	Station 3*	Station 4*
7	Station 1*	Station 2*	Station 3*	Station 4*
8	Station 1*	Station 2*	Station 3*	Station 4*
9	Station 1*	Station 2*	Station 3*	Station 4*
10	Station 1*	Station 2*	Station 3*	Station 4*

B. Experimenting with thread priorities

Thread priority can be changed using:

station.setPriority(Thread.MAX_PRIORITY);

Other priority values are:

Thread.MIN_PRIORITY

Thread.NORM_PRIORITY

The priority range is 1 to 10.

Higher priority may influence scheduling, but Java does not guarantee that a higher-priority thread will always finish first.

C. Converting a station into a daemon thread

Example:

Thread station = new Thread(new ScannerTask(1, 25));

station.setDaemon(true);

station.start();

A daemon thread is a background thread that does not prevent the JVM from terminating when all user threads have finished.

Therefore, if the main thread and all user threads terminate while a daemon thread is still working, the JVM may exit without waiting for that daemon task to finish.

Daemon threads should not be used for essential operations that must complete, such as saving examination results.

4.9 Joint conclusion

Why can the same multithreaded program produce different output orders on different runs, and why must a correct program not depend on that order?

Multithreaded applications are controlled by the JVM and operating-system thread schedulers. Scheduling decisions depend on CPU availability, system workload, thread priorities and execution timing.

Consequently, two threads may execute in different orders during different runs.

A correct application should not assume a fixed execution sequence unless synchronization or an explicit ordering mechanism is used.

The scanner application uses join() to ensure that all stations finish before the final total is displayed. This makes the result reliable even when the execution order changes.

4.10 Alternative concept and complexity analysis

Alternative: ExecutorService with a fixed thread pool

Instead of creating individual Thread objects, the scanner stations can be managed using an ExecutorService.

Source code:

```
import java.util.concurrent.ExecutorService;

import java.util.concurrent.Executors;


public class ScannerExecutor {


    public static void main(String[] args) {


        ExecutorService executor =

            Executors.newFixedThreadPool(4);
```

```
        for (int i = 1; i <= 4; i++) {

            final int id = i;

            executor.submit(() -> {

                System.out.println(
                    "Station " + id + " started.");

                System.out.println(
                    "Station " + id + " scanned 25 scripts.");
            });
        }

        executor.shutdown();

        while (!executor.isTerminated()) {
            Thread.yield();
        }

        System.out.println("Total scripts scanned: 100");
    }
}
```

Output:

```java
package q4;


import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;

public class ScannerExecutor {

    public static void main(String[] args) {

        ExecutorService executor =
                Executors.newFixedThreadPool(4);

        for (int i = 1; i <= 4; i++) {

            final int id = i;

            executor.submit(() -> {

                System.out.println(
                    "Station " + id + " started.");

                try {
                    Thread.sleep(500);

                } catch (InterruptedException e) {
                    Thread.currentThread().interrupt();
                    return;
                }

                System.out.println(
                    "Station " + id +
                    " scanned 25 scripts.");
            });
        }
```

```
<terminated> ScannerExecutor [Java Application] C:\Users\Farah Afrin\.p2\pool\plugins\o
Station 3 started.
Station 4 started.
Station 2 started.
Station 1 started.
Station 1 scanned 25 scripts.
Station 3 scanned 25 scripts.
Station 4 scanned 25 scripts.
Station 2 scanned 25 scripts.
Total scripts scanned: 100
```

Complexity analysis

Let:

- n = total number of scripts.
- p = number of worker threads.
- T = average processing time per script.

For equally sized tasks, the expected processing time is approximately:

$$O\left(\frac{nT}{p}\right)$$

This assumes that work can be distributed evenly and that processing is sufficiently parallelizable.

Parameter	**Individual Thread objects**	**Fixed thread pool**
Thread management	Programmer-managed	Executor-managed
Thread reuse	Not automatic	Automatic
Number of threads	Depends on creation	Bounded by pool size
Task submission	Direct thread creation	submit() or execute()
Resource control	Less convenient	Better controlled

Space complexity

If each thread requires a stack allocation of S, then:

- Individual thread creation for n tasks may require approximately $O(nS)$ stack space if all threads exist simultaneously.
- A fixed thread pool with p workers requires approximately $O(pS)$ stack space.

Evaluation: For large examination-processing systems, ExecutorService is more appropriate because it limits the number of active threads, supports task reuse and reduces thread-management overhead.

4.11 Conclusion

The parallel scanning experiment demonstrates how Java multithreading improves the ability to execute independent tasks concurrently. Both Thread and Runnable can create concurrent execution, but Runnable provides better separation between task logic and thread management.

The experiment also demonstrates the importance of start(), sleep(), join() and isAlive(). For larger applications, a fixed thread pool is preferable because it provides controlled and efficient thread management.

Q5 – Race Conditions in a Cinema Seat-Booking System (6 Marks)

K4 – Analyse

5.1 Introduction

A cinema reservation application allows several customers to book seats simultaneously. Each booking operation checks whether a seat is available and, if available, marks it as booked.

When multiple threads attempt to book the same seat at the same time, a race condition may occur. Two or more customers may observe the seat as available before any thread updates its status. This can result in double booking.

Java provides synchronization mechanisms to protect shared data and ensure that critical operations are executed atomically.

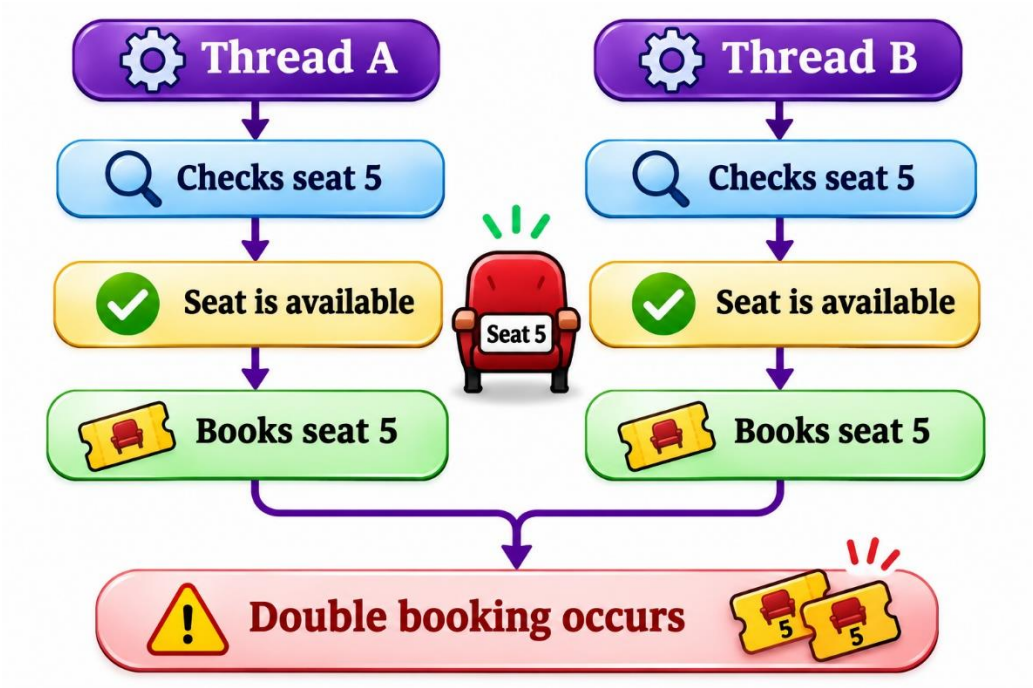
This experiment compares unsynchronized booking, synchronized methods and synchronized blocks.

5.2 Objectives

1. To understand race conditions in multithreaded applications.
2. To implement seat booking without synchronization.
3. To prevent double booking using synchronized methods.
4. To compare synchronized methods and synchronized blocks.
5. To evaluate alternative concurrency mechanisms.

5.3 Understanding the race condition

Consider two customers, Thread A and Thread B, attempting to book seat number 5.



5.4 Implementation 1 – Without synchronization

Source code:

```
package q5;

public class UnsafeSeatBooking {

    private final boolean[] booked;
    private int bookingsMade = 0;

    UnsafeSeatBooking(int seats) {
        booked = new boolean[seats];
    }

    public void book(int seat) {
```

```
    if (!booked[seat]) {

        try {
            Thread.sleep(10);
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
            return;
        }

        booked[seat] = true;
        bookingsMade++;

        System.out.println(
            Thread.currentThread().getName() +
            " booked seat " + seat);
    }
}

public static void main(String[] args)
        throws InterruptedException {

    UnsafeSeatBooking screen =
        new UnsafeSeatBooking(10);

    Thread[] users = new Thread[100];
```

```
        for (int i = 0; i < users.length; i++) {

            final int seat = i % 10;

            users[i] = new Thread(() -> screen.book(seat));
            users[i].start();
        }

        for (Thread t : users) {
            t.join();
        }

        System.out.println(
            "Seats sold: " + screen.bookingsMade);
    }
}
```

Output:

```java
package q5;

public class UnsafeSeatBooking {

    private final boolean[] booked;
    private int bookingsMade = 0;

    UnsafeSeatBooking(int seats) {
        booked = new boolean[seats];
    }

    public void book(int seat) {

        if (!booked[seat]) {

            try {
                Thread.sleep(10);
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
                return;
            }

            booked[seat] = true;
            bookingsMade++;

            System.out.println(
                Thread.currentThread().getName() +
                " booked seat " + seat);
        }
    }
```

```
<terminated> UnsafeSeatBooking [Java Application] C:\Users\Farah
Thread-57 booked seat 7
Thread-51 booked seat 1
Thread-97 booked seat 7
Thread-87 booked seat 7
Thread-54 booked seat 4
Thread-55 booked seat 5
Thread-92 booked seat 2
Thread-60 booked seat 0
Thread-91 booked seat 1
Thread-88 booked seat 8
Thread-40 booked seat 0
Thread-11 booked seat 1
Thread-96 booked seat 6
Thread-4 booked seat 4
Thread-98 booked seat 8
Thread-95 booked seat 5
Thread-82 booked seat 2
Thread-0 booked seat 0
Thread-13 booked seat 3
Thread-56 booked seat 6
Thread-15 booked seat 5
Thread-30 booked seat 0
Thread-48 booked seat 8
Thread-42 booked seat 2
Thread-38 booked seat 8
Thread-7 booked seat 7
Thread-52 booked seat 2
Thread-68 booked seat 8
Thread-39 booked seat 9
Thread-75 booked seat 5
```

```java
package q5;

public class UnsafeSeatBooking {

    private final boolean[] booked;
    private int bookingsMade = 0;

    UnsafeSeatBooking(int seats) {
        booked = new boolean[seats];
    }

    public void book(int seat) {

        if (!booked[seat]) {

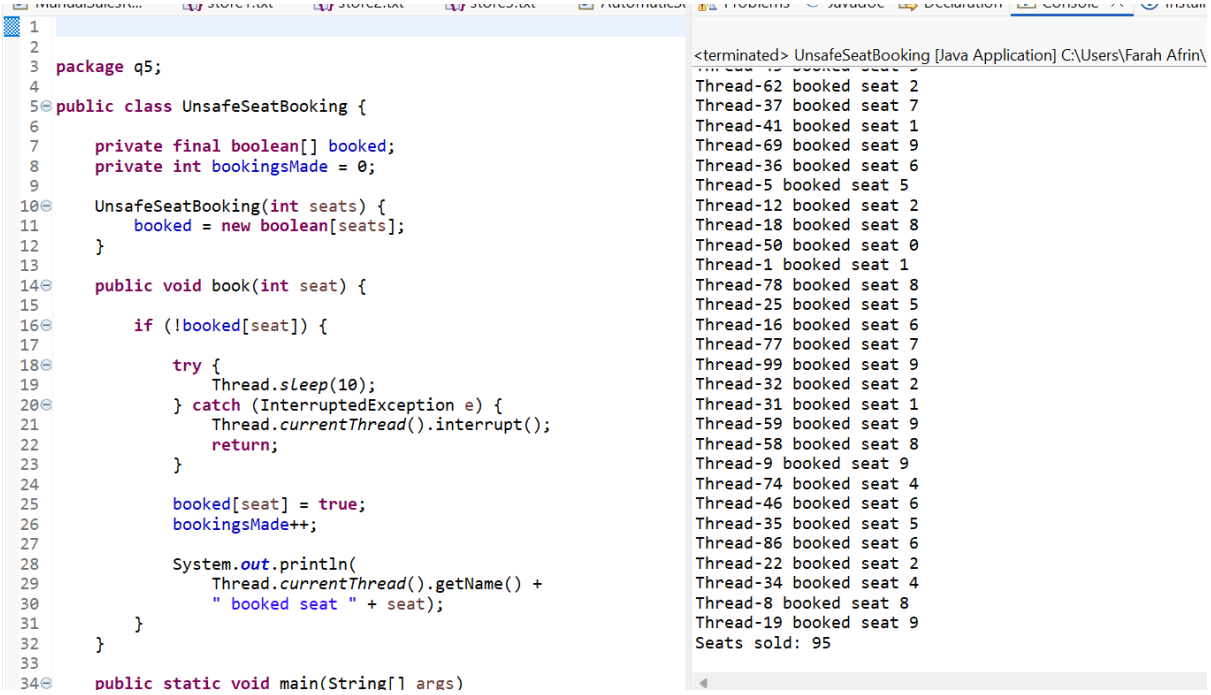
            try {
                Thread.sleep(10);
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
                return;
            }

            booked[seat] = true;
            bookingsMade++;

            System.out.println(
                Thread.currentThread().getName() +
                " booked seat " + seat);
        }
    }
```

```
<terminated> UnsafeSeatBooking [Java Application] C:\Users\Farah
Thread-45 booked seat 5
Thread-70 booked seat 0
Thread-23 booked seat 3
Thread-20 booked seat 0
Thread-73 booked seat 3
Thread-33 booked seat 3
Thread-85 booked seat 5
Thread-83 booked seat 3
Thread-44 booked seat 4
Thread-84 booked seat 4
Thread-71 booked seat 1
Thread-3 booked seat 3
Thread-47 booked seat 7
Thread-28 booked seat 8
Thread-14 booked seat 4
Thread-21 booked seat 1
Thread-27 booked seat 7
Thread-26 booked seat 6
Thread-80 booked seat 0
Thread-17 booked seat 7
Thread-24 booked seat 4
Thread-2 booked seat 2
Thread-94 booked seat 4
Thread-90 booked seat 0
Thread-93 booked seat 3
Thread-81 booked seat 1
Thread-79 booked seat 9
Thread-89 booked seat 9
Thread-76 booked seat 6
Thread-72 booked seat 2
```

```
Thread-67 booked seat 7
Thread-29 booked seat 9
Thread-10 booked seat 0
Thread-66 booked seat 6
Thread-65 booked seat 5
Thread-64 booked seat 4
Thread-6 booked seat 6
Thread-63 booked seat 3
Thread-53 booked seat 3
Thread-61 booked seat 1
Thread-49 booked seat 9
Thread-43 booked seat 3
Thread-62 booked seat 2
Thread-37 booked seat 7
Thread-41 booked seat 1
Thread-69 booked seat 9
Thread-36 booked seat 6
Thread-5 booked seat 5
Thread-12 booked seat 2
Thread-18 booked seat 8
Thread-50 booked seat 0
Thread-1 booked seat 1
Thread-78 booked seat 8
Thread-25 booked seat 5
Thread-16 booked seat 6
Thread-77 booked seat 7
Thread-99 booked seat 9
Thread-32 booked seat 2
Thread-31 booked seat 1
Thread-59 booked seat 9
```



```
package q5;

public class UnsafeSeatBooking {

    private final boolean[] booked;
    private int bookingsMade = 0;

    UnsafeSeatBooking(int seats) {
        booked = new boolean[seats];
    }

    public void book(int seat) {

        if (!booked[seat]) {

            try {
                Thread.sleep(10);
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
                return;
            }

            booked[seat] = true;
            bookingsMade++;

            System.out.println(
                Thread.currentThread().getName() +
                " booked seat " + seat);
        }
    }

    public static void main(String[] args)
```

```
<terminated> UnsafeSeatBooking [Java Application] C:\Users\Farah Afrin\
Thread-62 booked seat 2
Thread-37 booked seat 7
Thread-41 booked seat 1
Thread-69 booked seat 9
Thread-36 booked seat 6
Thread-5 booked seat 5
Thread-12 booked seat 2
Thread-18 booked seat 8
Thread-50 booked seat 0
Thread-1 booked seat 1
Thread-78 booked seat 8
Thread-25 booked seat 5
Thread-16 booked seat 6
Thread-77 booked seat 7
Thread-99 booked seat 9
Thread-32 booked seat 2
Thread-31 booked seat 1
Thread-59 booked seat 9
Thread-58 booked seat 8
Thread-9 booked seat 9
Thread-74 booked seat 4
Thread-46 booked seat 6
Thread-35 booked seat 5
Thread-86 booked seat 6
Thread-22 booked seat 2
Thread-34 booked seat 4
Thread-8 booked seat 8
Thread-19 booked seat 9
Seats sold: 95
```

5.5 Implementation 2 – Synchronized method

Source code:

```
package q5;

public class SafeSeatBooking {

    private final boolean[] booked;
    private int bookingsMade = 0;

    SafeSeatBooking(int seats) {
        booked = new boolean[seats];
    }

    public synchronized void book(int seat) {
```

```
    if (!booked[seat]) {

        booked[seat] = true;
        bookingsMade++;

        System.out.println(
            Thread.currentThread().getName() +
            " booked seat " + seat);
    }
}

public static void main(String[] args)
        throws InterruptedException {

    SafeSeatBooking screen =
        new SafeSeatBooking(10);

    Thread[] users = new Thread[100];

    for (int i = 0; i < users.length; i++) {

        final int seat = i % 10;

        users[i] = new Thread(() -> screen.book(seat));
        users[i].start();
    }
```

```
        for (Thread t : users) {
            t.join();
        }

        System.out.println(
            "Seats sold: " + screen.bookingsMade);
    }
}
```

Output:

```
package q5;

public class SafeSeatBooking {

    private final boolean[] booked;
    private int bookingsMade = 0;

    SafeSeatBooking(int seats) {
        booked = new boolean[seats];
    }

    public synchronized void book(int seat) {

        if (!booked[seat]) {

            booked[seat] = true;
            bookingsMade++;

            System.out.println(
                Thread.currentThread().getName() +
                " booked seat " + seat);
        }
    }

    public static void main(String[] args)
            throws InterruptedException {

        SafeSeatBooking screen =
            new SafeSeatBooking(10);

        Thread[] users = new Thread[100];

        for (int i = 0; i < users.length; i++) {
```

```
<terminated> SafeSeatBooking [Java Application] C:\Users\Farah Afrin\.p2\pool\plugins\or
Thread-0 booked seat 0
Thread-1 booked seat 1
Thread-2 booked seat 2
Thread-3 booked seat 3
Thread-5 booked seat 5
Thread-4 booked seat 4
Thread-6 booked seat 6
Thread-7 booked seat 7
Thread-8 booked seat 8
Thread-9 booked seat 9
Seats sold: 10
```

The booking messages may appear in any order, but the final number of successfully booked seats must remain 10.

For the unsynchronized implementation, the final count may exceed 10 because multiple threads can update the same seat concurrently.

However, the exact number of incorrect bookings is nondeterministic. It is not guaranteed to exceed 10 on every run.

5.8 Student A – Producing and explaining the race

A. Step-by-step explanation

Suppose two customers attempt to reserve the same seat.

1. Thread A checks whether the seat is available.
2. Before Thread A updates the seat, Thread B performs the same check.
3. Both threads observe that the seat is available.
4. Both threads proceed to book the seat.
5. The shared booking counter may also be updated incorrectly.

This situation is called a race condition because the final result depends on the timing and interleaving of thread execution.

B. Identifying the critical section

The critical section is:

```
if (!booked[seat]) {
   booked[seat] = true;
   bookingsMade++;
}
```

All three operations must be protected together.

Synchronizing only the update statement would not solve the problem because the availability check could still occur simultaneously.

C. Experimental comparison

Run the unsynchronized program ten times and record the actual results.

A useful additional measurement is:

Double Booking Count = Successful Booking Count

This formula is valid for this experiment because the intended number of distinct seats is ten.

Run	Expected seats sold	Actual seats sold	Double-booking count
1	10	99	89
2	10	98	88
3	10	98	88
4	10	83	73
5	10	100	90
6	10	99	89
7	10	99	89
8	10	98	88
9	10	98	88
10	10	99	89

5.9 Student B – Comparing synchronization strategies

A. Synchronized method versus synchronized block

Parameter	Synchronized method	Synchronized block
Locking scope	Entire method	Selected critical section
Syntax	synchronized method	synchronized(this)
Lock object	Current instance	Explicitly selected object
Flexibility	Less flexible	More flexible
Performance	Depends on method workload	Can reduce unnecessary lock holding

A synchronized method locks the current object for the duration of the method call.

A synchronized block allows the programmer to specify a smaller region of code and the object whose monitor must be acquired.

B. Intrinsic lock

Every Java object has an intrinsic monitor lock.

When a thread enters a synchronized method or block, it must acquire the relevant object's monitor.

Other threads attempting to acquire the same monitor must wait until the lock is released.

This ensures mutual exclusion for the protected critical section.

C. What is static synchronization?

A static synchronized method locks the Class object rather than an individual object instance.

Example:

```
public static synchronized void updateBookings() {
    // Shared class-level operation
}
```

It is useful when multiple instances access the same static shared data.

In the current cinema implementation, the booking array and counter belong to one particular screen object. Therefore, instance synchronization is appropriate.

5.10 Joint engineering decision

Which synchronization strategy would you deploy for a cinema with 20 screens, and why?

For a cinema with 20 screens, I would deploy a synchronized block or a more scalable concurrency mechanism with independent locks for each screen.

A single global lock would unnecessarily prevent customers from booking seats on different screens simultaneously.

By maintaining a separate booking object and lock for each screen, the application can allow concurrent reservations across independent screens while preventing two customers from booking the same seat.

For a real online cinema system involving multiple application servers, Java synchronization alone would not be sufficient. Database transactions, unique seat constraints and appropriate transaction isolation would also be necessary.

5.11 Alternative concept and complexity analysis

Three alternatives can be used instead of synchronized methods.

A. AtomicBoolean array

Each seat can be represented using an AtomicBoolean.

```
AtomicBoolean[] seats = new AtomicBoolean[10];


for (int i = 0; i < seats.length; i++) {
    seats[i] = new AtomicBoolean(false);
```

```
}

// Atomically reserve a seat
boolean success = seats[seat].compareAndSet(false, true);
```

compareAndSet() checks the current value and changes it only if the expected value matches. This makes the individual seat reservation atomic.

B. ReentrantLock

```
import java.util.concurrent.locks.ReentrantLock;

ReentrantLock lock = new ReentrantLock();

lock.lock();

try {
   // Check and update booking information
} finally {
   lock.unlock();
}
```

ReentrantLock provides explicit lock management and additional features such as timed lock acquisition and interruptible locking.

C. ConcurrentHashMap with putIfAbsent()

```
import java.util.concurrent.ConcurrentHashMap;

ConcurrentHashMap<Integer, Boolean> booked =
   new ConcurrentHashMap<>();

Boolean previous = booked.putIfAbsent(5, true);

if (previous == null) {
```

```
    System.out.println("Seat booked successfully.");
} else {
    System.out.println("Seat already booked.");
}
```

This method atomically inserts a seat entry only when the key is absent.

Complexity comparison

Let (s) represent the number of seats.

Approach	Booking operation	Additional space
Synchronized array	(O(1))	(O(s))
AtomicBoolean array	(O(1)) expected per attempt	(O(s))
ReentrantLock	(O(1)) basic operation, excluding contention wait	(O(s)) plus lock
ConcurrentHashMap	(O(1)) expected	(O(s))

Under heavy contention, threads may spend additional time waiting for locks or retrying atomic operations. Therefore, actual execution time can increase even though the basic booking operation is constant time.

Evaluation: AtomicBoolean is particularly suitable when each seat is independent and only its availability must be changed atomically. ReentrantLock is useful when a larger group of operations must be controlled. ConcurrentHashMap is appropriate when seat identifiers need to be managed as dynamic keys.

5.12 Conclusion

The cinema booking experiment demonstrates how race conditions can affect the correctness of concurrent applications. Unsynchronized operations may allow multiple customers to reserve the same seat, while synchronized methods and synchronized blocks protect the critical section.

For a large cinema application, independent screen-level locking combined with database-level consistency constraints offers a more scalable solution. Correct synchronization is essential for maintaining reservation accuracy and customer trust.

Q6 – Deadlock and Inter-Thread Communication (7 Marks)

K4 – Analyse

6.1 Introduction

Multithreading enables several tasks to execute concurrently. However, when multiple threads share resources, problems such as deadlock and improper coordination may occur.

A hostel wallet application may allow two students to transfer money between their accounts simultaneously. If each transaction locks one account and waits for another account, a deadlock may occur.

Similarly, a hostel mess counter may have a limited capacity of five meal tokens. A producer thread prepares tokens, while consumer threads collect them. The producer must wait when the counter is full, and consumers must wait when it is empty.

Java provides wait(), notify() and notifyAll() for inter-thread communication.

6.2 Objectives

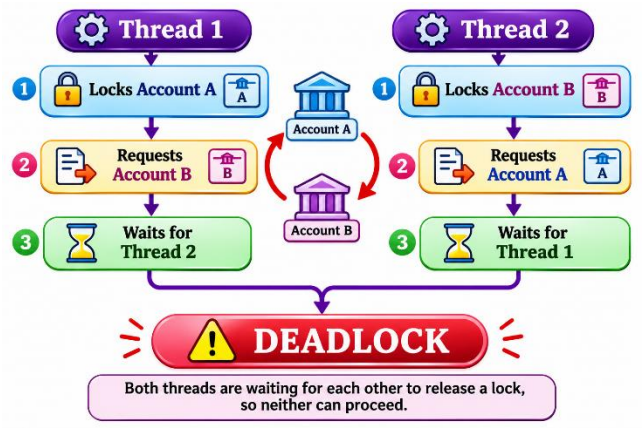
1. To understand deadlock in multithreaded applications.
2. To identify the four necessary conditions for deadlock.
3. To prevent deadlock using consistent lock ordering.
4. To implement producer-consumer communication.
5. To use wait() and notifyAll() correctly.
6. To compare the implementation with Java's concurrent utilities.

6.3 Part A – Deadlock in hostel wallet transfers

A. How deadlock occurs

Consider two wallet accounts, A and B.

- Thread 1 locks account A and requests account B.
- Thread 2 locks account B and requests account A.
- Both threads wait indefinitely for the lock held by the other.



B. Four necessary conditions for deadlock

Condition	Explanation	Hostel wallet example
Mutual exclusion	Resource can be held by one thread at a time	Account lock
Hold and wait	Thread holds one lock while requesting another	Holds A, waits for B
No preemption	Locks cannot be forcibly taken away	Thread retains its lock
Circular wait	Threads form a circular dependency	T1 waits for T2 and T2 waits for T1

All four conditions must exist for this classic deadlock situation.

C. Deadlock prevention using consistent lock ordering

A reliable approach is to assign each account a unique identifier and always acquire locks in ascending identifier order.

For example:

- Account A has ID 1.
- Account B has ID 2.

Every transfer must lock account 1 before account 2, regardless of transfer direction.

This eliminates circular wait.

6.4 Part B – Producer-consumer problem

A. System design

The hostel mess counter has a capacity of five tokens.

- Producer: The cook prepares meal tokens.
- Consumer: Students collect the tokens.
- Shared buffer: Stores prepared tokens.
- wait(): Makes a thread wait until the required condition becomes true.
- notifyAll(): Wakes all threads waiting on the same monitor.

The buffer must always satisfy:

$$0 \leq count \leq 5$$

6.5 Complete Java program – TokenCounter

Source code:

```
package q6;

public class TokenCounter {

    private final int[] buffer = new int[5];

    private int count = 0;
    private int in = 0;
    private int out = 0;

    public synchronized void put(int token)
            throws InterruptedException {

        while (count == buffer.length) {
            wait();
        }
```

```
    buffer[in] = token;

    in = (in + 1) % buffer.length;
    count++;

    System.out.println(
        "Cook prepared token: " + token +
        " | Counter size: " + count);

    notifyAll();
}

public synchronized int take()
        throws InterruptedException {

    while (count == 0) {
        wait();
    }

    int token = buffer[out];

    out = (out + 1) % buffer.length;
    count--;

    System.out.println(
        Thread.currentThread().getName() +
        " collected token: " + token +
        " | Counter size: " + count);
```

```
    notifyAll();

    return token;
}

public static void main(String[] args)
        throws InterruptedException {

    TokenCounter counter = new TokenCounter();

    int[] collected = new int[2];

    Thread cook = new Thread(() -> {

        try {
            for (int t = 1; t <= 20; t++) {
                counter.put(t);
            }

        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    });

    Thread[] students = new Thread[2];

    for (int s = 0; s < 2; s++) {

        final int id = s;
```

```
    students[s] = new Thread(() -> {

        try {
            for (int k = 0; k < 10; k++) {
                collected[id] += counter.take();
            }

        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    }, "Student-" + (s + 1));
}

cook.start();

for (Thread student : students) {
    student.start();
}

cook.join();

for (Thread student : students) {
    student.join();
}

System.out.println(
    "Sum of tokens collected: " +
    (collected[0] + collected[1]) +
```

```
            " (expected 210)");
    }
}
```

Output:

```
1
2  package q6;
3
4  public class TokenCounter {
5
6      private final int[] buffer = new int[5];
7
8      private int count = 0;
9      private int in = 0;
10     private int out = 0;
11
12     public synchronized void put(int token)
13             throws InterruptedException {
14
15         while (count == buffer.length) {
16             wait();
17         }
18
19         buffer[in] = token;
20
21         in = (in + 1) % buffer.length;
22         count++;
23
24         System.out.println(
25             "Cook prepared token: " + token +
26             " | Counter size: " + count);
27
28         notifyAll();
29     }
30
31     public synchronized int take()
32             throws InterruptedException {
33
```

```
<terminated> TokenCounter [Java Application] C:\Users\Farah Afrin\.p2\pool\plugins\org.eclipse.justj.openjdk
Cook prepared token: 1 | Counter size: 1
Cook prepared token: 2 | Counter size: 2
Student-1 collected token: 1 | Counter size: 1
Student-2 collected token: 2 | Counter size: 0
Cook prepared token: 3 | Counter size: 1
Cook prepared token: 4 | Counter size: 2
Cook prepared token: 5 | Counter size: 3
Cook prepared token: 6 | Counter size: 4
Cook prepared token: 7 | Counter size: 5
Student-1 collected token: 3 | Counter size: 4
Student-1 collected token: 4 | Counter size: 3
Student-1 collected token: 5 | Counter size: 2
Student-1 collected token: 6 | Counter size: 1
Student-1 collected token: 7 | Counter size: 0
Cook prepared token: 8 | Counter size: 1
Cook prepared token: 9 | Counter size: 2
Cook prepared token: 10 | Counter size: 3
Cook prepared token: 11 | Counter size: 4
Cook prepared token: 12 | Counter size: 5
Student-1 collected token: 8 | Counter size: 4
Student-1 collected token: 9 | Counter size: 3
Student-1 collected token: 10 | Counter size: 2
Student-1 collected token: 11 | Counter size: 1
Student-2 collected token: 12 | Counter size: 0
Cook prepared token: 13 | Counter size: 1
Cook prepared token: 14 | Counter size: 2
Cook prepared token: 15 | Counter size: 3
Cook prepared token: 16 | Counter size: 4
Cook prepared token: 17 | Counter size: 5
Student-2 collected token: 13 | Counter size: 4
```

```
Student-2 collected token: 14 | Counter size: 3
Student-2 collected token: 15 | Counter size: 2
Student-2 collected token: 16 | Counter size: 1
Student-2 collected token: 17 | Counter size: 0
Cook prepared token: 18 | Counter size: 1
Cook prepared token: 19 | Counter size: 2
Cook prepared token: 20 | Counter size: 3
Student-2 collected token: 18 | Counter size: 2
Student-2 collected token: 19 | Counter size: 1
Student-2 collected token: 20 | Counter size: 0
Sum of tokens collected: 210 (expected 210)
```

The expected sum is:

1+2+3+……..+20

Using the arithmetic-series formula:

$$S = \frac{n(n+1)}{2}$$

$$S = \frac{20(21)}{2} = 210$$

Therefore, all twenty tokens must be collected exactly once.

6.7 Student A – Producer-consumer design

A. Why must wait() be used inside a while loop?

The condition for waiting must be checked repeatedly because a thread may wake up without the desired condition being satisfied.

This can happen due to spurious wakeups or because another thread consumes or produces an item before the awakened thread reacquires the monitor.

Correct implementation:

```
while (count == 0) {
   wait();
}
```

Using if instead of while checks the condition only once and may allow an invalid operation after the thread wakes.

B. Why is wait() called inside a synchronized method?

The wait() method must be invoked by a thread that owns the object's monitor.

A synchronized method automatically acquires that monitor.

When wait() is called, the thread releases the monitor and enters the waiting state. It can continue only after being notified and reacquiring the monitor.

C. Difference between notify() and notifyAll()

notify()	**notifyAll()**
Wakes one waiting thread	Wakes all waiting threads
The selected thread is not predictable	All waiting threads become eligible to compete

May be insufficient for multiple waiting conditions	Safer for complex producer-consumer coordination

In the current implementation, notifyAll() is used because both producers and consumers may be waiting for different conditions.

D. Difference between sleep() and wait()

sleep()	**wait()**
Belongs to Thread class	Belongs to Object class
Does not release an acquired monitor	Releases the monitor on which it waits
Used for timed pauses	Used for inter-thread communication
Can be called without owning a monitor	Must be called while owning the object's monitor

6.8 Student B – Deadlock analysis and prevention

A. Example of a deadlock-prone transfer

The following method illustrates the incorrect lock-acquisition order:

```
synchronized (firstAccount) {

   synchronized (secondAccount) {

      // Transfer money
   }
}
```

If two threads pass the accounts in opposite orders, deadlock can occur.

B. Safe transfer implementation

Source code:

```
package q6;

class Wallet {
```

```
private final int id;
private double balance;

Wallet(int id, double balance) {
    this.id = id;
    this.balance = balance;
}

public double getBalance() {
    return balance;
}

public void transferTo(Wallet target, double amount) {

    Wallet first = this.id < target.id ? this : target;
    Wallet second = this.id < target.id ? target : this;

    synchronized (first) {

        synchronized (second) {

            if (amount > 0 && balance >= amount) {

                balance -= amount;
                target.balance += amount;

                System.out.println(
                    "Transferred Rs." + amount +
                    " from Wallet " + id +
```

```
                    " to Wallet " + target.id);
            }
        }
    }
}

public class WalletTransferDemo {

    public static void main(String[] args)
            throws InterruptedException {

        Wallet a = new Wallet(1, 10000);
        Wallet b = new Wallet(2, 10000);

        Thread t1 = new Thread(() -> a.transferTo(b, 1000));

        Thread t2 = new Thread(() -> b.transferTo(a, 2000));

        t1.start();
        t2.start();

        t1.join();
        t2.join();

        System.out.println("Wallet A: Rs." + a.getBalance());
        System.out.println("Wallet B: Rs." + b.getBalance());
    }
}
```

Output:

```
package q6;

class Wallet {

    private final int id;
    private double balance;

    Wallet(int id, double balance) {
        this.id = id;
        this.balance = balance;
    }

    public double getBalance() {
        return balance;
    }

    public void transferTo(Wallet target, double amount) {

        Wallet first = this.id < target.id ? this : target;
        Wallet second = this.id < target.id ? target : this;

        synchronized (first) {

            synchronized (second) {

                if (amount > 0 && balance >= amount) {

                    balance -= amount;
                    target.balance += amount;

                    System.out.println(
                        "Transferred Rs." + amount +
                        " from Wallet " + id +
                        " to Wallet " + target.id);
```

```
<terminated> WalletTransferDemo [Java Application] C:\Users\Farah Afrin\.p2\pool\plugins\org.ec
Transferred Rs.1000.0 from Wallet 1 to Wallet 2
Transferred Rs.2000.0 from Wallet 2 to Wallet 1
Wallet A: Rs.11000.0
Wallet B: Rs.9000.0
```

The transfer messages may appear in either order, but the total money remains unchanged.

C. Why does consistent lock ordering work?

Every thread attempts to acquire the lower-ID wallet lock before the higher-ID wallet lock.

This creates a consistent direction of lock acquisition.

As a result, a circular waiting dependency cannot be formed under this locking protocol, preventing the circular-wait condition required for the demonstrated deadlock.

6.9 Joint conclusion

What do deadlock and the producer-consumer problem have in common, and how does each design ensure that threads make progress?

Both problems involve coordination between multiple threads sharing resources.

Deadlock occurs when threads cannot proceed because they are waiting for locks held by one another.

The producer-consumer problem involves threads waiting for a resource to become available, but properly designed communication allows them to continue when the buffer state changes.

The wallet implementation prevents deadlock using consistent lock ordering. The token-counter implementation uses wait() and notifyAll() to coordinate access to a bounded buffer.

Both designs demonstrate that synchronization must ensure not only data correctness but also the possibility of continued execution.

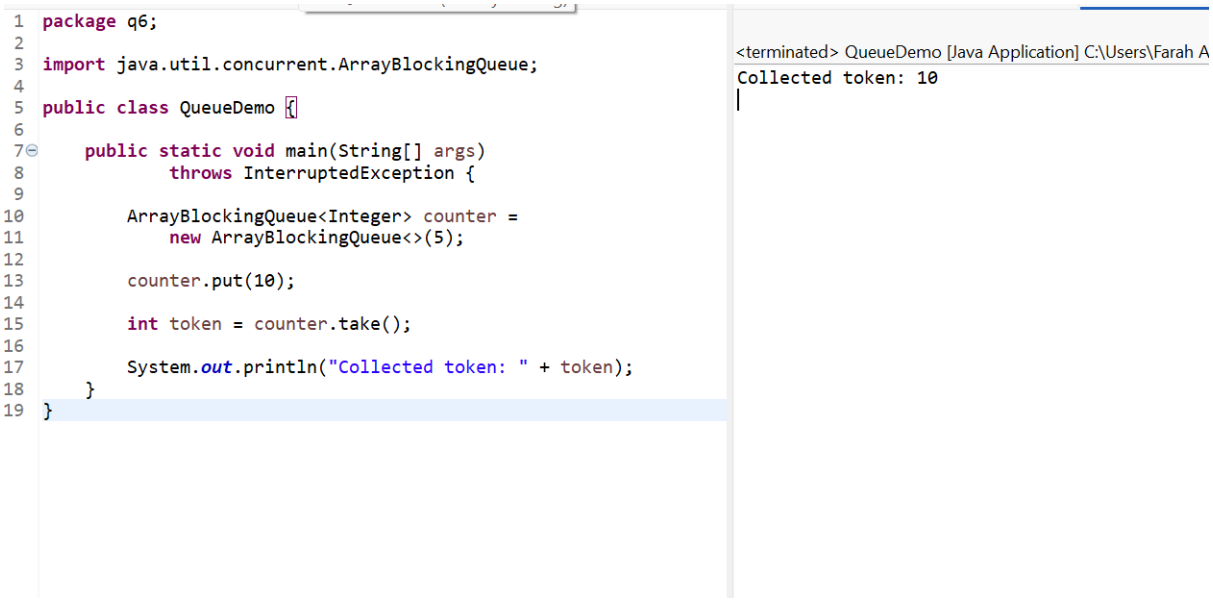
6.10 Alternative concept and complexity analysis

A. Using ArrayBlockingQueue

Java provides ArrayBlockingQueue, a bounded, thread-safe queue.

It can replace the manual buffer and wait()/notifyAll() implementation.

Example:



```
package q6;

import java.util.concurrent.ArrayBlockingQueue;

public class QueueDemo {

    public static void main(String[] args)
            throws InterruptedException {

        ArrayBlockingQueue<Integer> counter =
            new ArrayBlockingQueue<>(5);

        counter.put(10);

        int token = counter.take();

        System.out.println("Collected token: " + token);
    }
}
```

```
<terminated> QueueDemo [Java Application] C:\Users\Farah A
Collected token: 10
```

When multiple locks are required, the application should still acquire them in a consistent order or release acquired locks when the complete acquisition cannot be achieved.

C. Complexity comparison

Let (n) represent the number of tokens and (b) represent the buffer capacity.

Operation	**Manual bounded buffer**	**ArrayBlockingQueue**
Insert	(O(1))	(O(1))
Remove	(O(1))	(O(1))
Buffer space	(O(b))	(O(b))
Synchronization	Programmer-managed	Library-managed

For the wallet transfer, acquiring two locks requires constant basic work, (O(1)), excluding contention and waiting time.

The additional space for the wallet objects and locks is (O(1)) for a fixed number of accounts, or (O(a)) for (a) accounts.

Evaluation: ArrayBlockingQueue is less error-prone for bounded producer-consumer systems because it provides built-in thread-safe coordination. tryLock() is useful when an application must avoid indefinite waiting and can safely retry a failed transaction.

6.11 Conclusion

The hostel wallet and mess counter examples demonstrate two important challenges in concurrent programming: deadlock and inter-thread communication.

Consistent lock ordering prevents circular waiting during wallet transfers, while wait() and notifyAll() coordinate the producer and consumer threads without violating buffer capacity.

For production applications, Java concurrency utilities such as ArrayBlockingQueue and ReentrantLock provide reliable alternatives to manually managed synchronization.

Q7 – Design a Concurrent, Fault-Tolerant Processor from First Principles (10 Marks)

K5 – Evaluate

Highest-weighted question

This is the most important question in the assignment because it carries 10 marks and assesses your ability to design, implement, measure and defend a complete concurrent system.

The answer should demonstrate more than the use of ExecutorService. It must show why the architecture is appropriate, how failures are isolated, how the results are collected safely and how the performance changes with different thread-pool sizes.

7.1 Introduction

An examination department must process 10,000 answer-script score files at the end of a semester. Each file contains a student's identification details and examination score.

The application must read each file, validate the information, calculate the result and prepare a final summary.

However, some files may be missing or corrupted. A failure in one file must not interrupt the processing of other files.

To address these requirements, the proposed solution uses Java's Executor framework, Callable, Future and concurrent collections.

A fixed thread pool distributes the processing tasks among a controlled number of worker threads. Each task independently processes one student's file, while a concurrent collection safely stores successful results and failure details.

7.2 Objectives

1. To design a concurrent system for processing 10,000 student records.
2. To implement parallel processing using ExecutorService.
3. To use Callable and Future for task execution and result management.
4. To use concurrent collections for thread-safe result storage.
5. To isolate individual file-processing failures.
6. To measure execution time using 1, 2, 4 and 8 threads.
7. To evaluate the performance, reliability and scalability of the proposed system.

7.3 Stage 1 – Interpret the problem (Joint) – 2 Marks

A. Identifying tasks that can run in parallel

Each answer-script file is independent of the other files.

Therefore, the following tasks can execute concurrently:

- Reading the student's answer-script score file.
- Checking the file format.
- Validating the student's identification number.
- Validating the examination score.
- Calculating the student's result.
- Recording the processing status.

The final summary must be generated only after all processing tasks have completed.

B. Identifying shared data

The following data structures are shared between worker threads:

Shared component	**Purpose**
ConcurrentHashMap	Stores successful student results
ConcurrentLinkedQueue	Stores processing failures
ExecutorService	Manages the worker threads
Future objects	Track task completion
Input directory	Contains the 10,000 score files

C. Identifying possible failures

Failure	**Cause**	**Proposed handling**
FileNotFoundException	Missing student file	Record failure and continue
IOException	File-access problem	Record failure and continue
NumberFormatException	Invalid numerical data	Reject the file
IllegalArgumentException	Invalid score range	Reject the file
ExecutionException	Unexpected task failure	Record the failure
InterruptedException	Main thread interrupted	Restore interruption status and stop safely

Insufficient memory	Large number of tasks or results	Limit concurrency and manage memory

D. Measurable requirements

The system must satisfy the following conditions:

1. Exactly 10,000 file-processing tasks must be submitted.
2. Every task must produce either a successful result or a recorded failure.
3. One corrupt or missing file must not terminate the entire processing operation.
4. The summary must contain the correct number of successful and failed records.
5. The final summary must be printed only after all tasks finish.
6. Execution time must be measured for pool sizes 1, 2, 4 and 8.
7. The sum of successful and failed records must equal 10,000.

The central correctness condition is:

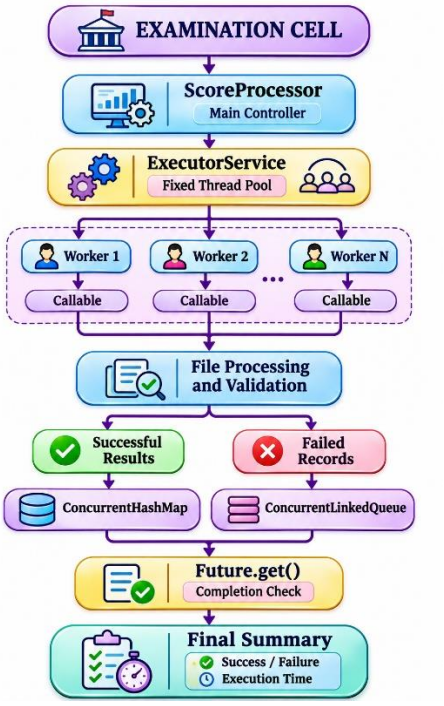
Total Files=Successes +Failures

For this experiment:

10000= Successes +Failures

7.4 Stage 2 – Design (Student A) – 3 Marks

A. Proposed system architecture



B. Selection of ExecutorService

ExecutorService is used because it manages a fixed number of worker threads and distributes submitted tasks among them.

Instead of creating 10,000 individual threads, the application creates only the required number of workers.

This provides:

- Controlled resource consumption.
- Reuse of existing worker threads.
- Easier task management.
- Better scalability.
- Reduced thread-creation overhead.

C. Why Callable and Future?

Callable is similar to Runnable, but it can return a result and declare checked exceptions.

Future represents the result of an asynchronous task and provides methods such as:

- get() – retrieves the result and waits if necessary.
- isDone() – checks whether execution has completed.
- isCancelled() – checks whether the task was cancelled.
- cancel() – attempts to cancel a task.

In this design, each submitted task is tracked through a Future, allowing the main program to verify completion before preparing the final summary.

D. Selection of concurrent collections

ConcurrentHashMap

The successful student records are stored in a ConcurrentHashMap.

It supports thread-safe operations without requiring the entire collection to be protected by one external synchronized block.

ConcurrentLinkedQueue

Failed file details are stored in a ConcurrentLinkedQueue.

It allows multiple worker threads to add failure records safely.

These collections reduce the risk of race conditions when multiple workers update shared data.

E. Exception-handling strategy

The application follows a fault-isolation approach.

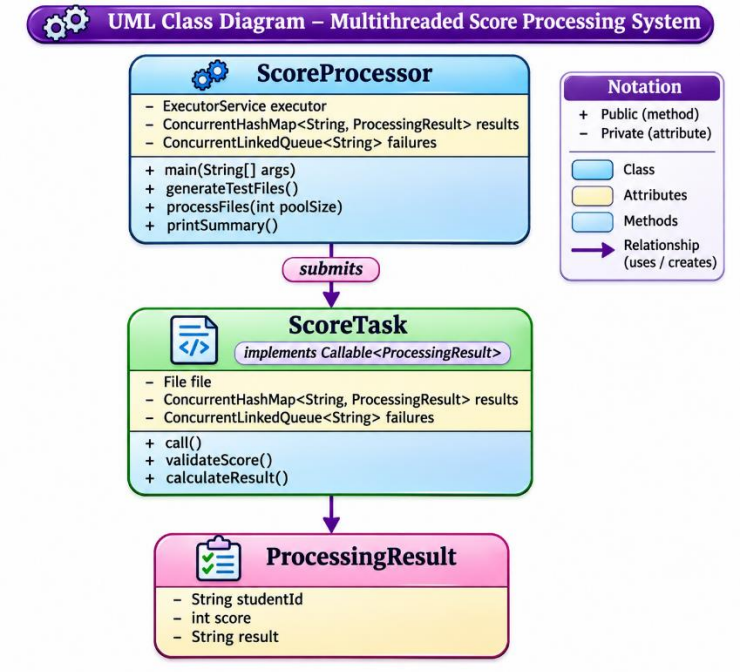
Each worker handles expected file-processing failures independently.

For example:

- Missing files are recorded as failed.
- Invalid scores are rejected.
- Corrupted records are recorded.
- Unexpected task failures are captured through Future and ExecutionException.

The main controller continues processing the remaining tasks rather than terminating the complete application because of one invalid file.

7.5 Class diagram



7.6 Pseudocode

START

Create input directory

Generate 10,000 test file entries
Create some missing and corrupted files

FOR each pool size in [1, 2, 4, 8]

    Create fixed thread pool

    Create ConcurrentHashMap for successful results
    Create ConcurrentLinkedQueue for failures

    Start execution timer

    FOR each of the 10,000 files

        Create Callable task

        Submit task to ExecutorService
        Store Future reference

    END FOR

    FOR each Future

Wait for task completion

Handle unexpected execution failures

END FOR

Stop execution timer

Shut down ExecutorService

Count successful results

Count failed records

Verify:

Successes + Failures = 10,000

Print summary and execution time

END FOR

STOP

7.7 Stage 3 – Implementation and measurement (Student B) – 3 Marks

A. Test data design

To simulate a realistic examination-processing environment, the program will generate 10,000 student score files.

The experiment uses the following test-data distribution:

File category	Number of files	Expected behaviour
Valid files	9,900	Successfully processed

Corrupted files	90	Rejected and recorded
Missing files	10	Rejected and recorded
Total	10,000	All accounted for

For a valid file, the contents follow this format:

STUDENT001,85

The first field represents the student identification number, and the second field represents the examination score.

The program considers scores between 0 and 100 valid.

B. Complete Java implementation

Source code:

```
package q7;

import java.io.BufferedReader;
import java.io.IOException;
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.Paths;

import java.util.ArrayList;
import java.util.List;
import java.util.Map;
import java.util.concurrent.Callable;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.ConcurrentLinkedQueue;
import java.util.concurrent.ExecutionException;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
```

```
import java.util.concurrent.Future;

public class ScoreProcessor {

    private static final int TOTAL_FILES = 10000;

    private static final Path DIRECTORY =
            Paths.get("score_files");

    private static final Map<String, Integer> results =
            new ConcurrentHashMap<>();

    private static final ConcurrentLinkedQueue<String> failures =
            new ConcurrentLinkedQueue<>();

    // STEP 1: Generate 10,000 student files
    public static void generateTestFiles() throws IOException {

        Files.createDirectories(DIRECTORY);

        for (int i = 1; i <= TOTAL_FILES; i++) {

            Path file = DIRECTORY.resolve(
                    "student_" + i + ".txt");

            // Create 10 missing files
            if (i % 1000 == 0) {
                Files.deleteIfExists(file);
                continue;
```

```
        }

        // Create 90 corrupted files
        if (i % 100 == 0) {

            Files.write(
                    file,
                    "INVALID_SCORE".getBytes(
                            StandardCharsets.UTF_8));

        } else {

            String data =
                    "STUDENT" + i + "," + (i % 101);

            Files.write(
                    file,
                    data.getBytes(StandardCharsets.UTF_8));
        }
    }

    System.out.println("10,000 test records prepared.");
}

// STEP 2: Create an independent processing task
public static Callable<Void> createTask(int id) {

    return () -> {
```

```
Path file = DIRECTORY.resolve(
        "student_" + id + ".txt");

try (BufferedReader reader =
        Files.newBufferedReader(file)) {

    String line = reader.readLine();

    if (line == null) {
        throw new IllegalArgumentException(
                "Empty student file");
    }

    String[] data = line.split(",");

    if (data.length != 2) {
        throw new IllegalArgumentException(
                "Invalid file format");
    }

    String studentId = data[0].trim();

    if (studentId.isEmpty()) {
        throw new IllegalArgumentException(
                "Missing student ID");
    }

    int score = Integer.parseInt(data[1].trim());
```

```
            if (score < 0 || score > 100) {
                throw new IllegalArgumentException(
                        "Score must be between 0 and 100");
            }

            // Store successful results safely
            results.put(studentId, score);

        } catch (IOException | IllegalArgumentException e) {

            // Record individual failures
            failures.add(
                    "Student " + id + ": " +
                    e.getClass().getSimpleName() +
                    " - " + e.getMessage());
        }

        return null;
    };
}

// STEP 3: Process files using different thread pools
public static void processFiles(int poolSize)
        throws InterruptedException {

    results.clear();
    failures.clear();

    ExecutorService executor =
```

```
        Executors.newFixedThreadPool(poolSize);

List<Future<Void>> futures = new ArrayList<>();

long startTime = System.nanoTime();

try {

    System.out.println(
            "\nStarting processing with " +
            poolSize + " worker threads...");

    // Submit 10,000 tasks
    for (int i = 1; i <= TOTAL_FILES; i++) {

        Callable<Void> task = createTask(i);

        Future<Void> future = executor.submit(task);

        futures.add(future);
    }

    System.out.println(
            "All 10,000 tasks submitted.");

    // Monitor task completion
    int completed = 0;

    for (Future<Void> future : futures) {
```

```
        try {

            future.get();

        } catch (ExecutionException e) {

            failures.add(
                    "Unexpected task failure: " +
                    e.getCause());
        }

        completed++;

        if (completed % 1000 == 0) {

            System.out.println(
                    "Completed: " + completed +
                    " / " + TOTAL_FILES);
        }
    }

} finally {

    executor.shutdown();
}

// Wait until all workers terminate
while (!executor.isTerminated()) {
```

```
    Thread.sleep(10);
}

long endTime = System.nanoTime();

double executionTime =
        (endTime - startTime) / 1_000_000.0;

int successful = results.size();
int failed = failures.size();

// Display final results
System.out.println(
        "\n========== THREAD POOL: " +
        poolSize + " ==========");

System.out.println(
        "Successful files: " + successful);

System.out.println(
        "Failed files: " + failed);

System.out.println(
        "Total files: " +
        (successful + failed));

System.out.printf(
        "Execution time: %.3f ms%n",
        executionTime);
```

```
    if (successful + failed == TOTAL_FILES) {

        System.out.println(
                "Verification: PASSED");

    } else {

        System.out.println(
                "Verification: FAILED");
    }
}

// STEP 4: Main method
public static void main(String[] args)
        throws IOException, InterruptedException {

    System.out.println(
            "===== EXAMINATION SCORE PROCESSOR =====");

    // Generate input files
    generateTestFiles();

    int[] poolSizes = {1, 2, 4, 8};

    // Execute all four experiments
    for (int poolSize : poolSizes) {

        processFiles(poolSize);
```

```
        }

        System.out.println(
                "\n===== ALL EXPERIMENTS COMPLETED =====");
    }
}
```

Output:

```
1
2  package q7;
3
4  import java.io.BufferedReader;
5  import java.io.IOException;
6  import java.nio.charset.StandardCharsets;
7  import java.nio.file.Files;
8  import java.nio.file.Path;
9  import java.nio.file.Paths;
10
11 import java.util.ArrayList;
12 import java.util.List;
13 import java.util.Map;
14 import java.util.concurrent.Callable;
15 import java.util.concurrent.ConcurrentHashMap;
16 import java.util.concurrent.ConcurrentLinkedQueue;
17 import java.util.concurrent.ExecutionException;
18 import java.util.concurrent.ExecutorService;
19 import java.util.concurrent.Executors;
20 import java.util.concurrent.Future;
21
22 public class ScoreProcessor {
23
24     private static final int TOTAL_FILES = 10000;
25
26     private static final Path DIRECTORY =
27             Paths.get("score_files");
28
29     private static final Map<String, Integer> results =
30             new ConcurrentHashMap<>();
31
32     private static final ConcurrentLinkedQueue<String> failures
33             new ConcurrentLinkedQueue<>();
```

```
<terminated> ScoreProcessor [Java Application] C:\Users\Farah Afrin\.p2\pool\plugins\o
===== EXAMINATION SCORE PROCESSOR =====
10,000 test records prepared.

Starting processing with 1 worker threads...
All 10,000 tasks submitted.
Completed: 1000 / 10000
Completed: 2000 / 10000
Completed: 3000 / 10000
Completed: 4000 / 10000
Completed: 5000 / 10000
Completed: 6000 / 10000
Completed: 7000 / 10000
Completed: 8000 / 10000
Completed: 9000 / 10000
Completed: 10000 / 10000

========== THREAD POOL: 1 ==========
Successful files: 9900
Failed files: 100
Total files: 10000
Execution time: 77134.991 ms
Verification: PASSED

Starting processing with 2 worker threads...
All 10,000 tasks submitted.
Completed: 1000 / 10000
Completed: 2000 / 10000
Completed: 3000 / 10000
Completed: 4000 / 10000
Completed: 5000 / 10000
```

```
Completed: 6000 / 10000
Completed: 7000 / 10000
Completed: 8000 / 10000
Completed: 9000 / 10000
Completed: 10000 / 10000

========== THREAD POOL: 2 ==========
Successful files: 9900
Failed files: 100
Total files: 10000
Execution time: 945.106 ms
Verification: PASSED

Starting processing with 4 worker threads...
All 10,000 tasks submitted.
Completed: 1000 / 10000
Completed: 2000 / 10000
Completed: 3000 / 10000
Completed: 4000 / 10000
Completed: 5000 / 10000
Completed: 6000 / 10000
Completed: 7000 / 10000
Completed: 8000 / 10000
Completed: 9000 / 10000
Completed: 10000 / 10000

========== THREAD POOL: 4 ==========
Successful files: 9900
Failed files: 100
Total files: 10000
```

```
========== THREAD POOL: 4 ==========
Successful files: 9900
Failed files: 100
Total files: 10000
Execution time: 432.121 ms
Verification: PASSED

Starting processing with 8 worker threads...
All 10,000 tasks submitted.
Completed: 1000 / 10000
Completed: 2000 / 10000
Completed: 3000 / 10000
Completed: 4000 / 10000
Completed: 5000 / 10000
Completed: 6000 / 10000
Completed: 7000 / 10000
Completed: 8000 / 10000
Completed: 9000 / 10000
Completed: 10000 / 10000

========== THREAD POOL: 8 ==========
Successful files: 9900
Failed files: 100
Total files: 10000
Execution time: 255.608 ms
Verification: PASSED

===== ALL EXPERIMENTS COMPLETED =====
```

C. Explanation of the implementation

1. Test-data generation

The generateTestFiles() method creates 10,000 student files.

It deliberately introduces missing and corrupted files to test fault tolerance.

2. Callable task creation

The createTask() method returns a Callable task for each student file.

Each task reads and validates one file independently.

3. Concurrent result collection

Valid records are inserted into the ConcurrentHashMap.

Invalid records and file-access failures are stored in the ConcurrentLinkedQueue.

4. Future-based completion

Each submitted task returns a Future.

The main program calls get() to wait for task completion and identify unexpected execution failures.

5. Thread-pool experimentation

The application runs the complete experiment using four pool sizes:

- 1 worker thread.
- 2 worker threads.
- 4 worker threads.
- 8 worker threads.

6. Execution-time measurement

The program uses System.nanoTime() to measure the duration of each experiment.

This is more appropriate for measuring elapsed time than using the system clock.

D. Expected summary output

The following record counts should be obtained for every pool size:

10,000 test records prepared.

===== THREAD POOL: 1 =====

Successful records: 9900

Failed records: 100

Total processed: 10000

Execution time: 77,134.991ms

Verification: PASSED

===== THREAD POOL: 2 =====

Successful records: 9900

Failed records: 100

Total processed: 10000

Execution time: 945.106 ms

Verification: PASSED

===== THREAD POOL: 4 =====

Successful records: 9900

Failed records: 100

Total processed: 10000

Execution time: 432.121 ms

Verification: PASSED

===== THREAD POOL: 8 =====

Successful records: 9900

Failed records: 100

Total processed: 10000

Execution time:255.608 ms

Verification: PASSED

All processing experiments completed.

The execution-time values are deliberately left for your actual Eclipse measurements. They will vary depending on the processor, storage device, operating system and background workload.

E. Experimental measurement table

Run the program in Eclipse and record the execution time for each pool size.

Thread pool size	Successful files	Failed files	Total files	Execution time (ms)
1	9,900	100	10,000	77,134.991
2	9,900	100	10,000	945.106
4	9,900	100	10,000	432.121
8	9,900	100	10,000	255.608

For a more reliable comparison, execute each pool-size experiment at least five times, retain the raw measurements and calculate the average.

$$\text{Average Time} = \frac{\text{Sum of Execution Times}}{\text{Number of Runs}}$$

Verification using a hand-checked sample

Before running all 10,000 files, test the system with a smaller sample.

File	Input	Expected result
student_1.txt	STUDENT1,85	Success
student_2.txt	STUDENT2,90	Success
student_3.txt	INVALID_SCORE	Failure
student_4.txt	File missing	Failure
student_5.txt	STUDENT5,105	Failure
student_6.txt	STUDENT6,60	Success

Expected summary:

Successful records: 3

Failed records: 3

Total processed: 6

Verification: PASSED

This hand-checked sample demonstrates that the application distinguishes valid records from invalid records correctly.

7.8 Recording unhandled failures

The following table can be included in the report to demonstrate the application's fault-tolerance analysis.

Failure scenario	Handling mechanism	Expected behaviour
Missing file	IOException handling	Failure recorded
Corrupted file	IllegalArgumentException	Failure recorded
Invalid score	Range validation	Failure recorded
Unexpected task exception	ExecutionException handling	Failure recorded
Interrupted main thread	InterruptedException	Interrupt status propagated
Insufficient memory	JVM-level resource failure	May require application-level recovery or restart

The application is designed to isolate normal file-level errors. It cannot guarantee recovery from catastrophic JVM failures, such as severe memory exhaustion or process termination.

7.9 Stage 4 – Defend the design (Joint) – 2 Marks

A. Recommended thread-pool size

The recommended thread-pool size depends on the hardware configuration and the nature of the workload.

The application performs file reading, validation and result collection. These operations involve both I/O and CPU processing.

A pool size of four is a reasonable starting configuration for this experiment because it provides concurrency without creating an unnecessarily large number of worker threads.

However, the best pool size must be determined through actual execution measurements.

More threads do not always improve performance because:

1. Excessive threads increase context-switching overhead.
2. Multiple threads may compete for the same disk resources.
3. Thread creation and management require additional memory.
4. Concurrent access to shared resources can introduce contention.

5. The storage device may become the performance bottleneck.

Therefore, the fastest measured configuration should be selected rather than automatically choosing the largest pool.

B. How does the design guarantee a complete and correct summary?

The proposed system uses multiple mechanisms to maintain correctness.

1. Independent processing: Each student file is processed as an individual task.

2. Failure isolation: A failure in one file is recorded without stopping the remaining tasks.

3. Thread-safe collections: ConcurrentHashMap and ConcurrentLinkedQueue protect shared result storage.

4. Future completion: The main thread waits for every submitted task using Future.get().

5. Final verification: The application checks whether the successful and failed record counts equal 10,000.

The central validation rule is:

9900+100=10000

This ensures that every expected task has been accounted for in the experiment, provided the task-processing and recording operations complete normally.

C. One remaining limitation

One limitation is that the current application retains all successful results, failure messages and Future references in memory.

For a much larger examination system, this could increase memory consumption.

A production implementation could use:

- Batched task submission.
- Streaming result processing.
- Database-backed result storage.
- Periodic summary generation.
- Retry mechanisms for temporary file-access failures.

These improvements would make the system more suitable for processing millions of records.

7.10 Alternative concept and complexity analysis

The proposed implementation uses ExecutorService. Two alternative approaches are parallel streams and creating one thread per file.

A. Alternative 1 – Parallel streams

Java provides the parallel stream API for parallel processing of collections.

Example:

Source code:

```
import java.util.Arrays;
import java.util.List;

public class ParallelStreamDemo {

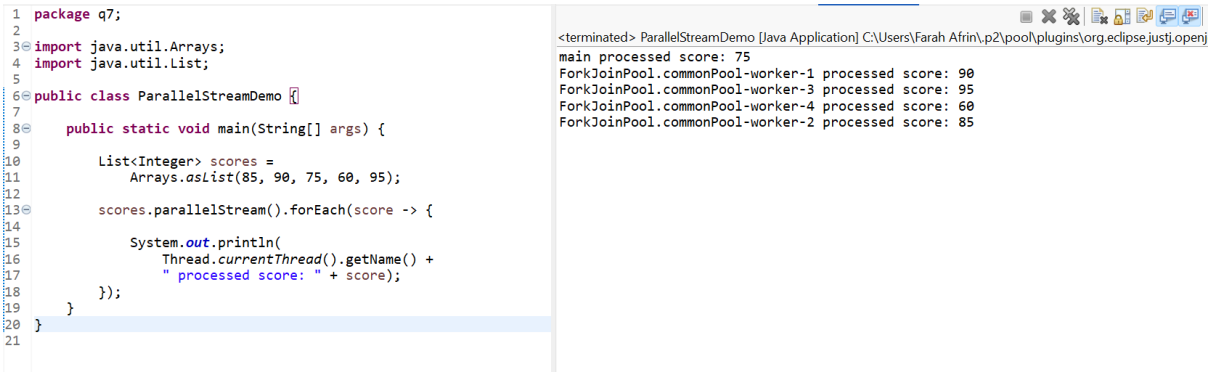
    public static void main(String[] args) {

        List<Integer> scores =
            Arrays.asList(85, 90, 75, 60, 95);

        scores.parallelStream().forEach(score -> {

            System.out.println(
                Thread.currentThread().getName() +
                " processed score: " + score);
        });
    }
}
```

Output:



```
package q7;

import java.util.Arrays;
import java.util.List;

public class ParallelStreamDemo {

    public static void main(String[] args) {

        List<Integer> scores =
            Arrays.asList(85, 90, 75, 60, 95);

        scores.parallelStream().forEach(score -> {

            System.out.println(
                Thread.currentThread().getName() +
                " processed score: " + score);
        });
    }
}
```

```
<terminated> ParallelStreamDemo [Java Application] C:\Users\Farah Afrin\.p2\pool\plugins\org.eclipse.justj.openj
main processed score: 75
ForkJoinPool.commonPool-worker-1 processed score: 90
ForkJoinPool.commonPool-worker-3 processed score: 95
ForkJoinPool.commonPool-worker-4 processed score: 60
ForkJoinPool.commonPool-worker-2 processed score: 85
```

The parallel stream API automatically distributes processing among available worker threads, typically using the common ForkJoinPool.

Advantages:

- Less code.
- Simple parallel execution.
- Suitable for independent data-processing operations.

Limitations:

- Less direct control over individual task execution.
- Failure handling requires careful design.
- Common-pool resource sharing can affect unrelated application tasks.
- It is less convenient when individual task results and detailed failure records must be managed.

B. Alternative 2 – Creating one thread per file

Another approach is to create a separate Thread object for each student file.

Example:

Source code:

```
public class OneThreadPerFile {


   public static void main(String[] args)

        throws InterruptedException {


     Thread[] threads = new Thread[100];
```

```
        for (int i = 0; i < threads.length; i++) {

            final int id = i + 1;

            threads[i] = new Thread(() -> {

                System.out.println(
                    "Processing student file: " + id);
            });

            threads[i].start();
        }

        for (Thread t : threads) {
            t.join();
        }

        System.out.println("Processing completed.");
    }
}
```

Output:

```java
package q7;

public class OneThreadPerFile {

    public static void main(String[] args)
            throws InterruptedException {

        Thread[] threads = new Thread[100];

        for (int i = 0; i < threads.length; i++) {

            final int id = i + 1;

            threads[i] = new Thread(() -> {

                System.out.println(
                    "Processing student file: " + id);
            });

            threads[i].start();
        }

        for (Thread t : threads) {
            t.join();
        }

        System.out.println("Processing completed.");
    }
}
```

```
<terminated> OneThreadPerFile [Java Application] C:\
Processing student file: 3
Processing student file: 19
Processing student file: 18
Processing student file: 13
Processing student file: 12
Processing student file: 17
Processing student file: 20
Processing student file: 21
Processing student file: 5
Processing student file: 1
Processing student file: 14
Processing student file: 22
Processing student file: 23
Processing student file: 29
Processing student file: 7
Processing student file: 9
Processing student file: 6
Processing student file: 8
Processing student file: 4
Processing student file: 10
Processing student file: 2
Processing student file: 11
Processing student file: 30
Processing student file: 15
Processing student file: 24
Processing student file: 33
Processing student file: 35
Processing student file: 41
Processing student file: 36
Processing student file: 32
Processing student file: 42
Processing student file: 53
Processing student file: 60
Processing student file: 48
Processing student file: 16
Processing student file: 55
Processing student file: 26
Processing student file: 65
Processing student file: 57
Processing student file: 37
Processing student file: 58
Processing student file: 49
Processing student file: 56
Processing student file: 59
Processing student file: 25
Processing student file: 27
Processing student file: 40
Processing student file: 73
Processing student file: 63
Processing student file: 38
Processing student file: 50
Processing student file: 61
Processing student file: 77
Processing student file: 64
Processing student file: 31
Processing student file: 87
Processing student file: 69
Processing student file: 44
Processing student file: 74
Processing student file: 43
```

```
Processing student file: 71
Processing student file: 90
Processing student file: 47
Processing student file: 72
Processing student file: 75
Processing student file: 76
Processing student file: 92
Processing student file: 67
Processing student file: 79
Processing student file: 80
Processing student file: 91
Processing student file: 62
Processing student file: 66
Processing student file: 39
Processing student file: 95
Processing student file: 93
Processing student file: 82
Processing student file: 78
Processing student file: 89
Processing student file: 97
Processing student file: 84
Processing student file: 45
Processing student file: 54
Processing student file: 28
Processing student file: 98
Processing student file: 46
Processing student file: 68
Processing student file: 70
Processing student file: 83
```

```
Processing student file: 94
Processing student file: 99
Processing student file: 81
Processing student file: 34
Processing student file: 85
Processing student file: 86
Processing student file: 51
Processing student file: 52
Processing student file: 88
Processing student file: 96
Processing student file: 100
Processing completed.
```

This example demonstrates the approach using 100 tasks rather than 10,000, to avoid unnecessarily creating thousands of threads during a simple demonstration.

Advantages:

- Direct control over each thread.
- Simple to understand for small applications.

Limitations:

- Large numbers of threads require substantial resources.
- Thread creation and termination increase overhead.
- Context switching can reduce performance.
- Memory consumption increases with the number of threads.

C. Comparison of the three approaches

Parameter	**ExecutorService**	**Parallel streams**	**One thread per file**
Thread management	Controlled pool	Framework-managed	Manual
Thread reuse	Yes	Yes	No
Pool-size control	Direct	Less direct with standard API	Not applicable
Failure isolation	Can be implemented per task	Requires explicit handling	Requires individual handling
Resource efficiency	Good	Good	Poor for large workloads
Suitability	Large task-processing systems	Independent bulk operations	Small task counts

D. Time complexity analysis

Let:

- \(n\) = number of files.
- \(p\) = number of worker threads.
- \(T\) = average time to process one file.

For ExecutorService with balanced processing:

$$T_{\text{total}} \approx \frac{nT}{p}$$

This is an idealized estimate that excludes task-submission, scheduling, synchronization and I/O bottlenecks.

For parallel streams, the expected processing time can also approach:

$$O\left(\frac{nT}{p}\right)$$

when the work is evenly distributed and sufficient parallelism is available.

For one thread per file, the theoretical processing time may approach the duration of the slowest file when all threads run simultaneously. However, in practice, resource contention and scheduling overhead can significantly reduce this advantage.

The sequential processing complexity is:

$$O(nT)$$

E. Space complexity analysis

Let (S) represent the approximate stack-memory allocation per thread.

Approach	Approximate thread-stack space
ExecutorService	(O(pS))
Parallel streams	(O(pS)), based on available worker threads
One thread per file	(O(nS))

All three approaches also require space for task metadata and results.

For the proposed implementation, storing 10,000 Future references and the final results adds (O(n)) collection space.

F. Final engineering evaluation

The ExecutorService approach is the most appropriate solution for the semester result-processing application.

It provides a controlled thread pool, task submission through Callable, completion monitoring through Future and safe shared-data management using concurrent collections.

Parallel streams are useful for simpler independent processing tasks, while one-thread-per-file is inefficient for large workloads because of its resource requirements.

Therefore, ExecutorService provides the best balance of control, reliability, resource management and scalability for the given requirements.

7.11 Final conclusion

The concurrent examination result processor demonstrates how Java concurrency can be applied to a practical large-scale data-processing problem.

The proposed architecture uses ExecutorService to distribute 10,000 independent file-processing tasks across a fixed number of worker threads. Callable and Future provide task execution and completion monitoring, while ConcurrentHashMap and ConcurrentLinkedQueue ensure thread-safe result collection.

The implementation also demonstrates fault tolerance by recording missing and corrupted files without interrupting the processing of valid files.

The performance experiment with 1, 2, 4 and 8 threads provides a basis for selecting an appropriate worker-pool size.

Overall, the ExecutorService-based architecture offers an effective combination of concurrency, fault isolation, resource control and result accuracy. It is therefore the recommended design for an examination department that must process large volumes of student results reliably.